AY 2025-26

III Semester

CSMC-209: Python Programming

LAB FILE

SUBMITTED BY

Naman Rai (24247)

To

Dr. Shatrughan Modi
(Faculty School of Computing)

SCHOOL OF ELECTRONICS

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY UNA

SALOH, HIMACHAL PRADESH -177209

LIST OF EXPERIMENTS

S.No	Name of the Experiment	Date	Marks
1.	Check the number entered from the console is even or odd.		
2.	Print “Welcome to Python world”.		
3.	Convert KM into miles and vice-versa.		
4.	Swap two numbers without using third variable.		
5.	Calculate area of a Circle, Rectangle, Square		
6.	Calculate Simple interest and compound interest for the principal		
7.	Find the nature of roots of quadratic equation.		
8.	Display Grade of student according to his/her marks using if else ladder.		
9.	Converts the temperature taken by user in Fahrenheit to Celsius.		
10.	The total numbers of days are given and you have to convert that in perfect no of days, year and month.		
11.	Same as the above program but here instead of days the seconds are given and you need to convert it into hour, min and seconds’ format.		
12.	Check the type of triangle (isosceles, equilateral, scalene) when the side lengths are entered through console.		
13.	Find largest as well as smallest of four numbers entered through console by user.		
14.	Calculate volume of the sphere when radius is given in double from the console.		
15.	Calculate factorial of a number entered from the console.		
16.	Display the reverse of number entered through keyboard. also display the sum of digits of the same number.		
17.	To display the mathematical tables for the first n natural numbers		
18.	Calculate the value of e^x, sin(x) using series.		
19.	Program to check whether the number entered through console is a. Palindrome or not b. Prime or not c. Armstrong or not		

20.	Program to generate following series up to n no. of terms (n is entered through console). a. Fibonacci b. Geometric terms where first term is 1 and common ratio is 1⁄2.		
21.	Program to generate following output for n number of rows(n is entered through console). a. Upper Right Triangle for character '&' b. Lower Right Triangle for character '*' c. Pyramid for any character entered through keyboard. d. Pascal Triangle		
22.	Program to find sum of series up to n terms (n is entered through console). a. 1+3+5+7..... b. 1 - 1/2 + 1/3 - 1/4 + 1/5 ..... c. 1! + 2! + 3! + 4! ..... d. 1^2 +2^ 2+3^ 2+..... e. 1^3+2^3+3^3+.....		
		Total Marks	

SIGNATURE OF THE STUDENT **SIGNATURE OF THE FACULTY**

EXPERIMENT-01

Objective: Check the number entered from the console is even or odd.
Source Code:

Code:

```
# Program to check whether a number is even or odd
 x = int(input("Enter a number: "))   # input


if (x & 1) == 1:  # computation
    print(str(x) + " is odd number")  # output
else:
     print(str(x) + " is even number")  # output
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Ass
ignment\Python-Lab-Assignments\Lab Assignments 1\Q1.py"
Enter a number: 7
7 is odd number
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Ass
ignment\Python-Lab-Assignments\Lab Assignments 1\Q1.py"
Enter a number: 18
18 is even number
```

EXPERIMENT-02

Objective: Print “Welcome to Python world”.

Code:

```
# introduction to the python language

print("welcome to the python world")
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Ass
ignment\Python-Lab-Assignments\Lab Assignments 1\Q2.py"
Welcome to Python World
```

Experiment-03

Objective: Convert KM into miles and vice-versa.

Code:

```
# Program to convert km to miles and miles to km


km_to_miles_factor = 0.621371

miles_to_km_factor = 1.60934


km = float(input("Enter the number of kilometres: "))  # input

miles = float(input("Enter the number of miles: "))    # input


km_in_miles = km * km_to_miles_factor  # computation

miles_in_km = miles * miles_to_km_factor  # computation


print(str(km) + " kilometres is equal to " + str(km_in_miles) + "
miles.")  # output

print(str(miles) + " miles is equal to " + str(miles_in_km) + "
kilometres.")  # output
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Deskt



                                                   python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Ass
ignment\Python-Lab-Assignments\Lab Assignments 1\Q3.py"
Enter the number of kilometers: 5
Enter the number of miles: 8
5.0 kilometers is equal to 3.106855 miles.
8.0 miles is equal to 12.87472 kilometers.
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Ass
ignment\Python-Lab-Assignments\Lab Assignments 1\Q3.py"
Enter the number of kilometers: 10
Enter the number of miles: 18
10.0 kilometers is equal to 6.21371 miles.
18.0 miles is equal to 28.96812 kilometers.
```

Experiment-04

Objective: Swap two numbers without using third variable.

Code:

```
# Program to swap two numbers without using third variable

a = int(input("Enter first number: "))  # input
b = int(input("Enter second number: "))  # input

a = a + b  # computation
b = a - b
a = a - b

print("After swapping, a =", a, "and b =", b)  # output
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q4.py"
Enter first number: 100000
Enter second number: 68
After swapping, a = 68 and b = 100000
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q4.py"
Enter first number: 34687097
Enter second number: 4326546758968
After swapping, a = 4326546758968 and b = 34687097
```

Experiment-05

Objective: Calculate area of a Circle, Rectangle, Square

Code:

```
# Area of circle


r = float(input("Enter radius: "))  # input

area_circle = 3.1416 * r * r  # computation

print("Area of circle:", area_circle)  # output
```

```
# Area of rectangle

l = float(input("Enter length: "))  # input
w = float(input("Enter width: "))  # input
area_rectangle = l * w  # computation
print("Area of rectangle:", area_rectangle)  # output

# Area of square

s = float(input("Enter side: "))  # input
area_square = s * s  # computation
print("Area of square:", area_square)  # output
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q5.py"
Enter radius: 57
Area of circle: 10207.0584
Enter length: 76
Enter width: 59
Area of rectangle: 4484.0
Enter side: 89
Area of square: 7921.0
```

Experiment-06

Objective: Calculate Simple interest and compound interest for the principal.

Code:

```
# Program to calculate simple and compound interest

p = float(input("Enter principal amount: "))  # input
r = float(input("Enter rate of interest: "))  # input
t = float(input("Enter time in years: "))  # input

si = (p * r * t) / 100  # computation
amount = p * (1 + r / 100) ** t
ci = amount - p  # computation
```

```
print("Simple Interest is:", si)  # output
print("Compound Interest is:", ci)  # output
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q6.py"
Enter principal amount: 50000
Enter rate of interest: 14
Enter time in years: 3
Simple Interest is: 21000.0
Compound Interest is: 24077.200000000026
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q6.py"
Enter principal amount: 150000
Enter rate of interest: 10
Enter time in years: 20
Simple Interest is: 300000.0
Compound Interest is: 859124.9923988417
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Des
ktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignments 1\Q6.py"
Enter principal amount: 6700000
Enter rate of interest: 13
Enter time in years: 25
Simple Interest is: 21775000.0
Compound Interest is: 135544633.22164756
```

Experiment-07

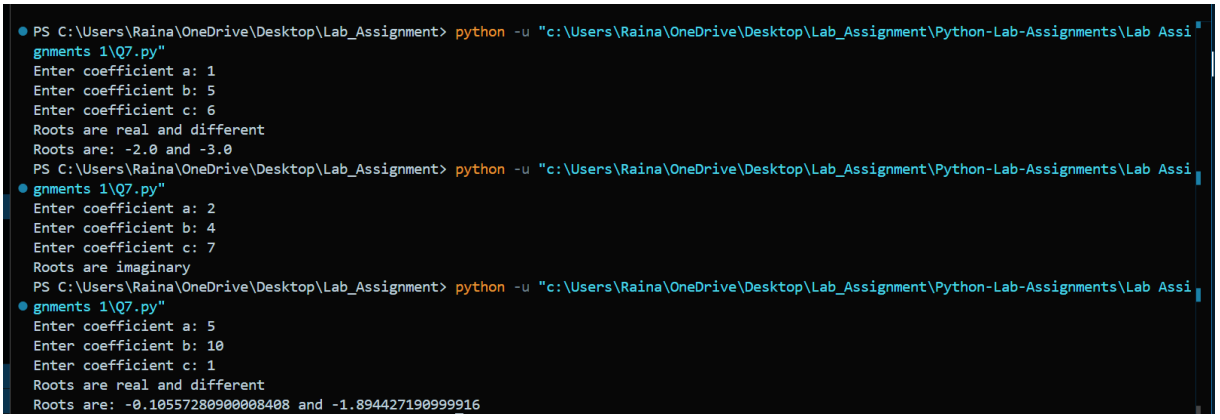
Objective: Find the nature of roots of quadratic equation.

Code:

```
# Program to find nature and roots of quadratic equation


a = float(input("Enter coefficient a: "))  # input
b = float(input("Enter coefficient b: "))  # input
c = float(input("Enter coefficient c: "))  # input



d = b**2 - 4*a*c  # computation
```

```
if d > 0:
    print("Roots are real and different")  # output
elif d == 0:
    print("Roots are real and equal")  # output
else:
    print("Roots are imaginary")  # output


if d >= 0:
    root1 = (-b + d**0.5) / (2*a)
    root2 = (-b - d**0.5) / (2*a)
    print("Roots are:", root1, "and", root2)  # output
```

Output:



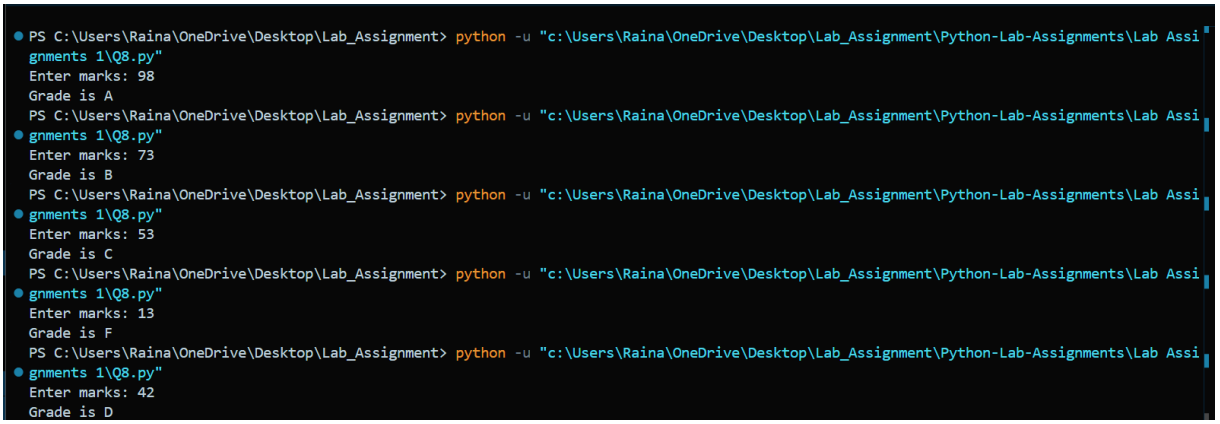
Experiment-08

Objective: Display Grade of student according to his/her marks using if else ladder.

Code:

```
# Program to display grade using if-else ladder
marks = int(input("Enter marks: "))  # input
if marks > 80:
    grade = "A"
elif marks >= 61:
    grade = "B"
elif marks >= 51:
    grade = "C"
elif marks >= 41:
    grade = "D"
elif marks >= 35:
    grade = "E"
else:
    grade = "F"
print("Grade is", grade)
```

Output:



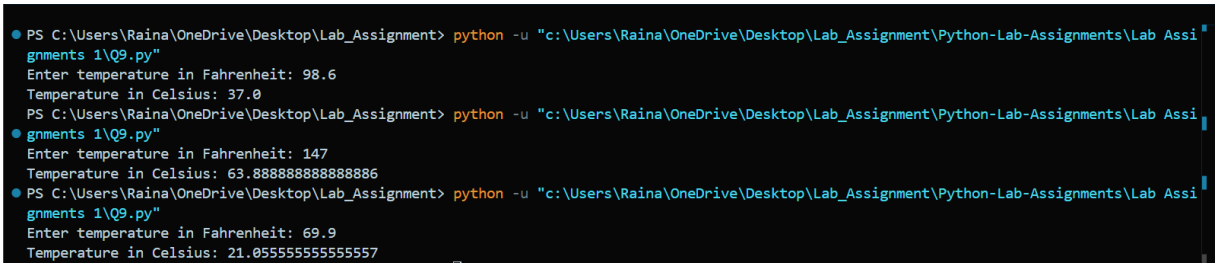
Experiment-09

Objective: Converts the temperature taken by user in Fahrenheit to Celsius.

Code:

```
#Create a program which converts the temperature taken by user in
Fahrenheit to Celsius.


# Input

f = float(input("Enter temperature in Fahrenheit: "))


# Computation

c = (f - 32) * 5 / 9


# Output

print("Temperature in Celsius:", c)
```

Output:



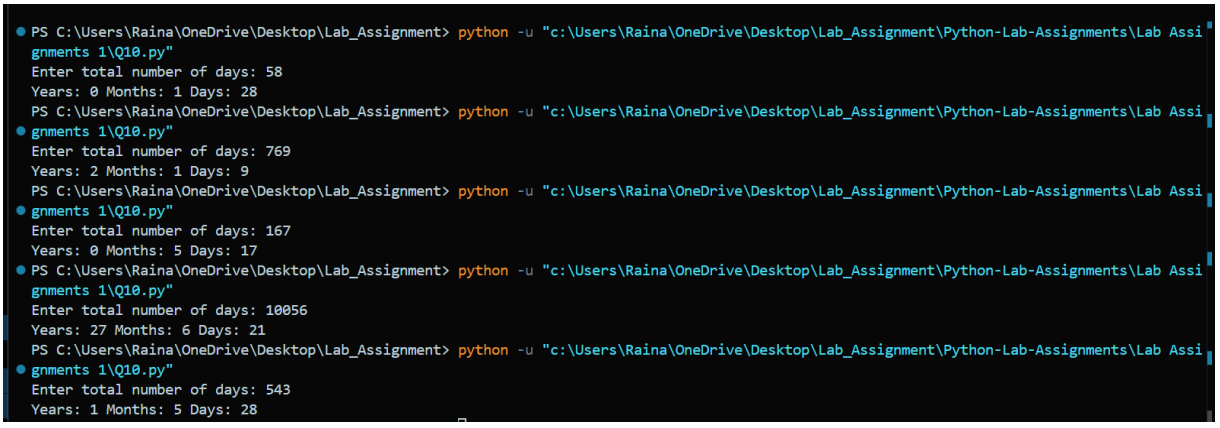
EXPERIMENT-10

Objective: The total numbers of days are given and you have to convert that in perfect no of days, year and month.

Code:

```
# Convert total days into years, months, and days


days = int(input("Enter total number of days: "))  # input


years = days // 365  # computation
months = (days % 365) // 30
remaining_days = (days % 365) % 30


print("Years:", years, "Months:", months, "Days:", remaining_days)
```

Output:



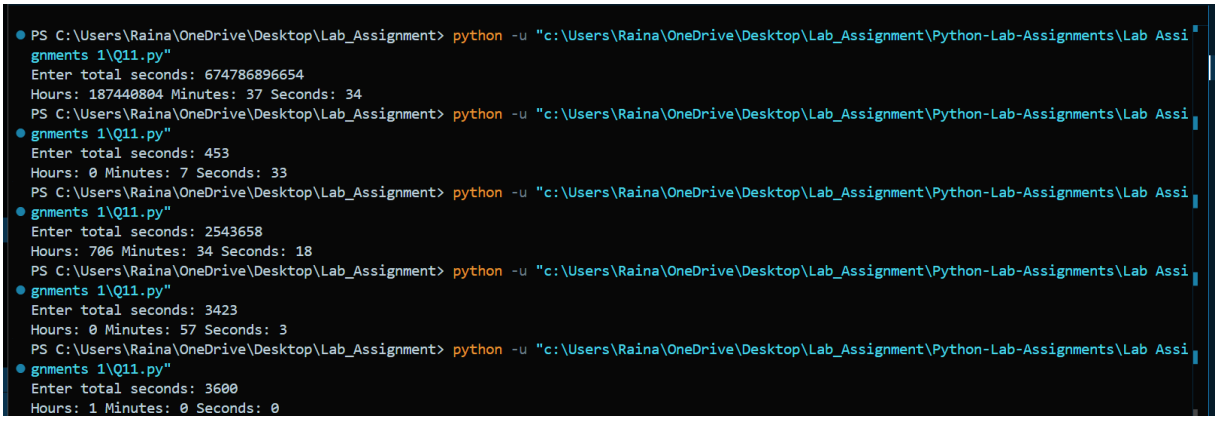
Experiment-11

Objective: Same as the above program but here instead of days the seconds are given and you need to convert it into hour, min and seconds' format.

Code:

```
# Convert seconds into hours, minutes, and seconds


seconds = int(input("Enter total seconds: "))  # input


hours = seconds // 3600  # computation

minutes = (seconds % 3600) // 60

remaining_seconds = seconds % 60


print("Hours:", hours, "Minutes:", minutes, "Seconds:",
remaining_seconds)
```

Output:



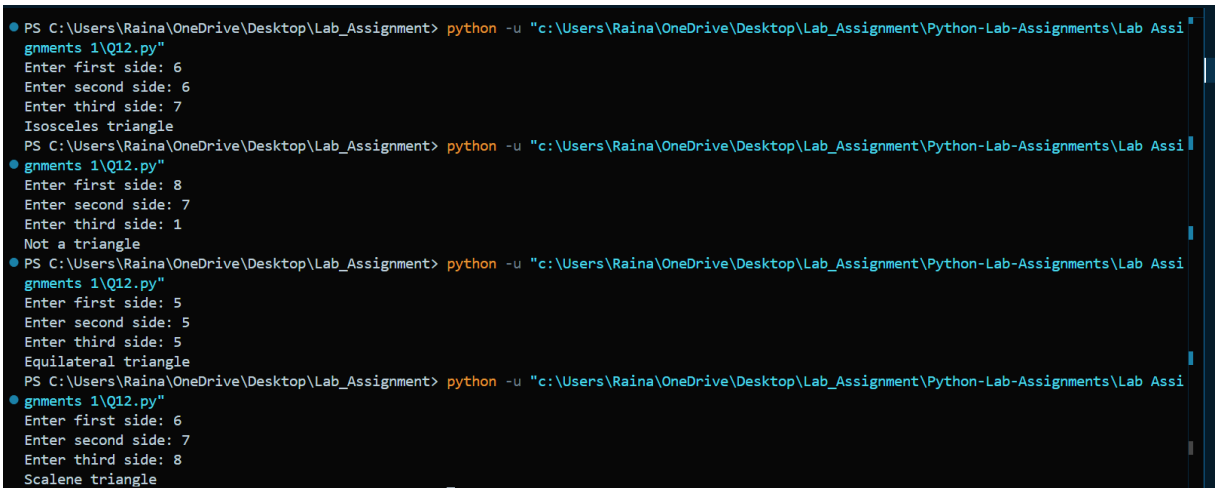
Experiment-12

Objective: Check the type of triangle (isosceles, equilateral, scalene) when the side lengths are entered through console.

Code:

```
a = float(input("Enter side a: "))
b = float(input("Enter side b: "))
c = float(input("Enter side c: "))


if a == b == c:
    print("Equilateral triangle")
elif a == b or b == c or a == c:
    print("Isosceles triangle")
else:
    print("Scalene triangle")
```

Output:



EXPERIMENT-13

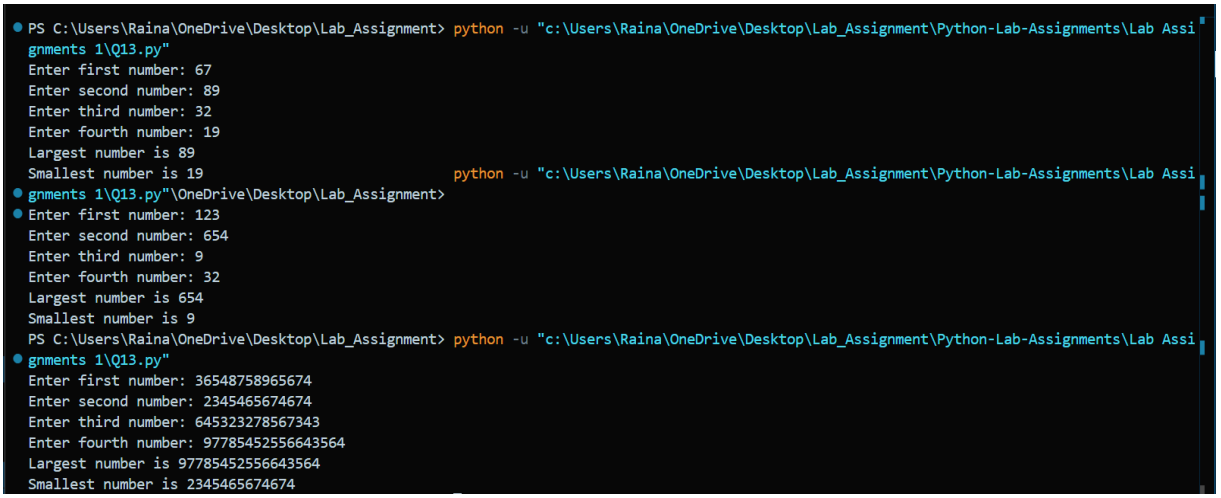
Objective: Find largest as well as smallest of four numbers entered through console by user.

Code:

```
# Take four numbers as input from the user
a = float(input("Enter number 1: "))
b = float(input("Enter number 2: "))
c = float(input("Enter number 3: "))
d = float(input("Enter number 4: "))

# Assume the first number is the largest initially
largest = a

# Compare with second number
if b > largest:
    largest = b

# Compare with third number
if c > largest:
    largest = c

# Compare with fourth number
if d > largest:
    largest = d

# Assume the first number is the smallest initially
smallest = a
```

```
# Compare with second number
if b < smallest:
    smallest = b
# Compare with third number
if c < smallest:
    smallest = c

# Compare with fourth number
if d < smallest:
    smallest = d

# Print the results
print("Largest number =", largest)
print("Smallest number =", smallest)
```

Output:



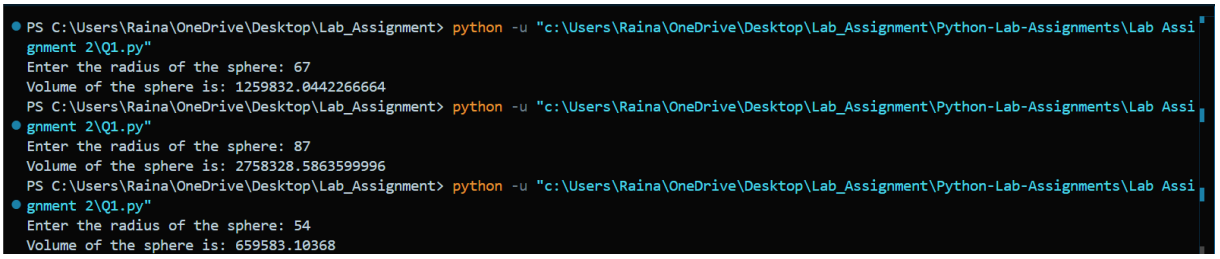
Experiment-14

Objective: Calculate volume of the sphere when radius is given in double from the console.

Code:

```
# input - take radius of sphere from user
radius = float(input("Enter the radius of the sphere: "))

# computation - apply volume formula (4/3) * pi * r^3
volume = (4 / 3) * 3.14159 * radius ** 3

# output - print the volume of the sphere
print("Volume of the sphere is:", volume)
```

Output:



Experiment-15

Objective: Calculate factorial of a number entered from the console.

Code:

```
# input - take number from user
n = int(input("Enter a number: "))

# computation - calculate factorial using loop
fact = 1
for i in range(1, n+1):
    print(fact,i)
```

```
    fact *= i


# output - display factorial
print("Factorial of", n, "is", fact)
```

Output:

Experiment-16

Objective: Display the reverse of number entered through keyboard. also display the sum of digits of the same number

Code:

```
# input - take number from user
num = int(input("Enter a number: "))


# computation - reverse number and calculate digit sum
rev = 0
sum_digits = 0
temp = num
while temp > 0:
    digit = temp % 10
    rev = rev * 10 + digit
    sum_digits += digit
    temp //= 10
```

```
# output - display reversed number and digit sum
print("Reversed number:", rev)
print("Sum of digits:", sum_digits)
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q3.py"
Enter a number: 89
Reversed number: 98
Sum of digits: 17
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q3.py"
Enter a number: 49
Reversed number: 94
Sum of digits: 13
```

Experiment-17

Objective: To display the mathematical tables for the first n natural numbers.

Code:

```
# input - take n from user
n = int(input("Enter the value of n: "))

# computation - generate tables from 1 to n
for i in range(1, n+1):
    print("Table of", i)
    for j in range(1, 11):
        print(i, "x", j, "=", i*j)
    print()
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q4.py"
Enter the value of n: 2
Table of 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10

Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

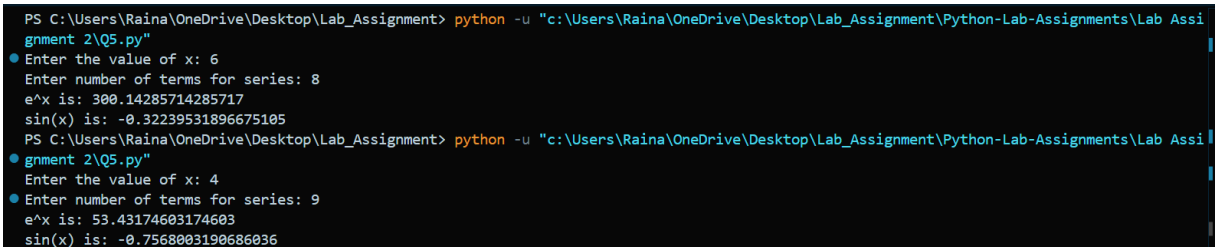
Experiment-18

Objective: Calculate the value of e^x, sin(x) using series.

Code:

```
# input - take x from user
x = float(input("Enter the value of x: "))
n = int(input("Enter number of terms for series: "))

# computation - calculate e^x using series
e_pow_x = 0
for i in range(n):
    e_pow_x += x**i / math.factorial(i)

# computation - calculate sin(x) using series
sin_x = 0
for i in range(n):
    sign = (-1)**i
    sin_x += sign * (x**(2*i + 1)) / math.factorial(2*i + 1)

# output - display e^x and sin(x)
print("e^x is:", e_pow_x)
print("sin(x) is:", sin_x)
```

Output:



Experiment-19

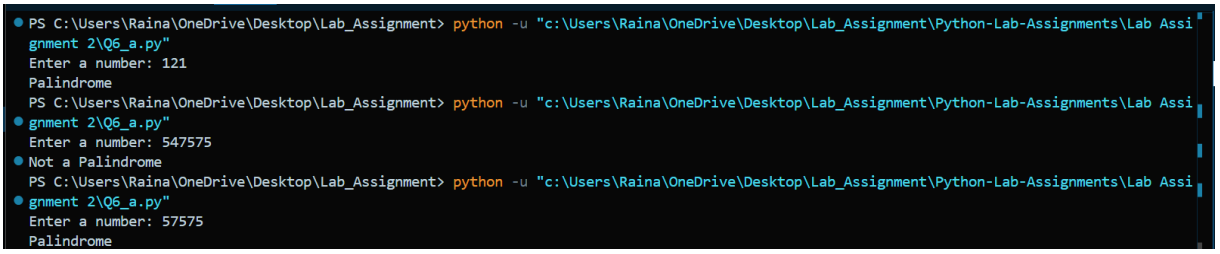
Objective: Program to check whether the number entered through console is

a. Palindrome or not b. Prime or not c. Armstrong or not

Code: (a) Palindrome

```
# input - take number from user
num = int(input("Enter a number: "))

# computation - reverse number
rev = 0
temp = num
while temp > 0:
    digit = temp % 10
    rev = rev * 10 + digit
    temp //= 10

# condition - check palindrome
if num == rev:
    print("Palindrome")
else:
    print("Not a Palindrome")
```

Output:



(b) Prime or not

```
# input - take number from user
n = int(input("Enter a number: "))


# condition - check for prime
if n < 2:
    print("Not Prime")
else:
    is_prime = True
    for i in range(2, n):
        if n % i == 0:
            is_prime = False
            break
    if is_prime:
        print("Prime")
    else:
        print("Not Prime")
```

Output:

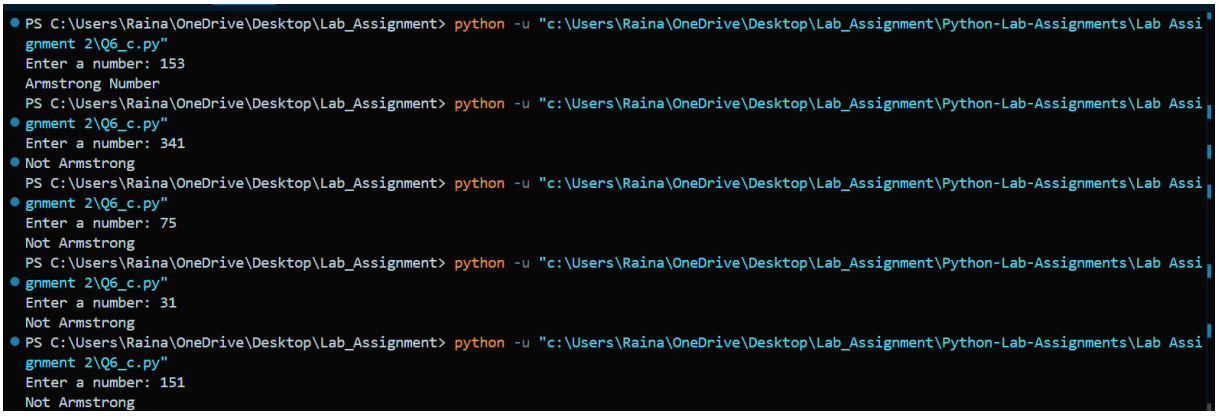
```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q6_b.py"
Enter a number: 19
Prime
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> 13
13
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q6_b.py"
Enter a number: 675675675
Not Prime
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q6_b.py"
Enter a number: 78906543219
Not Prime
```

(c) Armstrong or not

```
# Question 6c: Check if number is Armstrong.


# input - take number from user
num = int(input("Enter a number: "))
```

```
# computation - calculate Armstrong sum
sum_pow = 0
temp = num
order = len(str(num))
while temp > 0:
    digit = temp % 10
    sum_pow += digit ** order
    temp //= 10

# condition - check Armstrong
if num == sum_pow:
    print("Armstrong Number")
else:
    print("Not Armstrong")
```

Output:



Experimental-20

Objective: Program to generate following series up to n no. of terms (n is enter through

console).

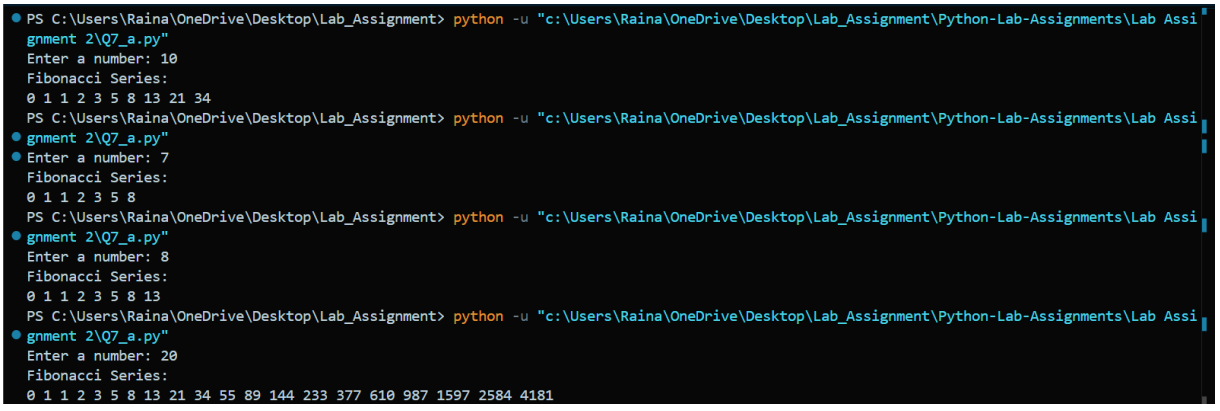
a. Fibonacci

b. Geometric terms where first term is 1 and common ratio is 1⁄2.

Code:(a) Fibonacci

```
# input - take number of terms from user
n = int(input("Enter a number: "))

# computation - generate Fibonacci terms
def fib(n):
    if n == 0 or n == 1:
        return n            # base case: 0 or 1
    else:
        return fib(n - 1) + fib(n - 2)   # recursive case

# output - print Fibonacci series
print("Fibonacci Series:")
for i in range(n):
    print(fib(i), end=" ")
```

Output:

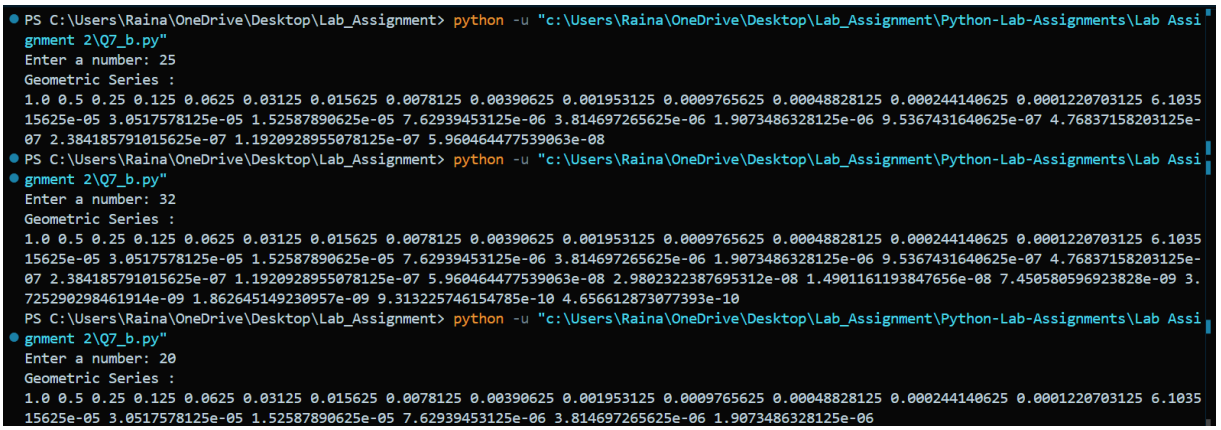


(b) Geometric terms where first term is 1 and common ratio is 1⁄2.

```
# input - take number of terms from user
n = int(input("Enter a number: "))

# computation - generate Fibonacci terms
```

```
def geometric_series(i):
    if(n==0):
        return 1
    a=1
    R=1/2
    return a*R**i


# output - print Geometric series
print("Geometric Series :")
for i in range(n):
    print(geometric_series(i),end=" ")
```

Output:



Experiment-21

Objective: Program to generate following output for n number of rows(n is entered through

console).

a. Upper Right Triangle for character '&'

b. Lower Right Triangle for character '*'

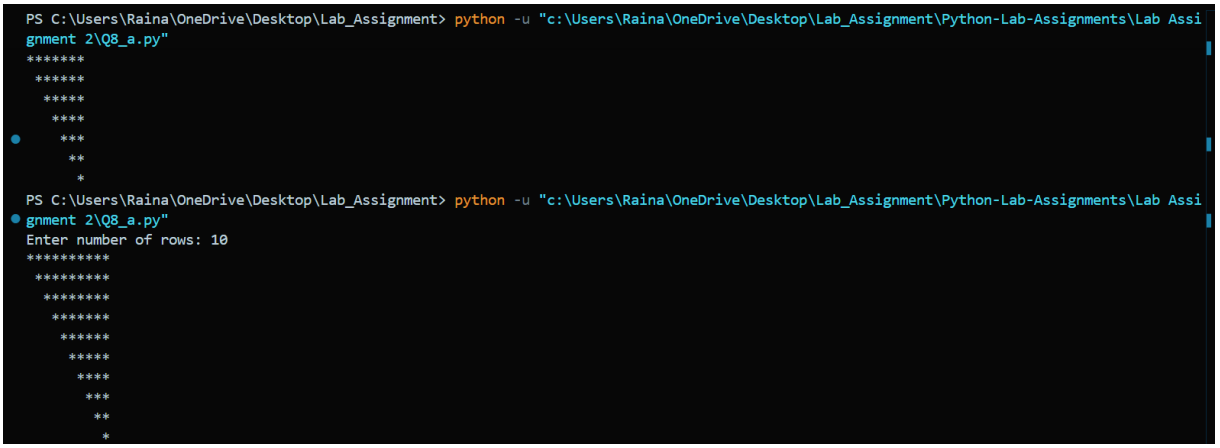
c. Pyramid for any character entered through keyboard.

d. Pascal Triangle

Code: (a) Upper Right Triangle for character '*'

```
# input - take number of rows from user
n = int(input("Enter number of rows: "))

# output - print the pattern
for i in range(n):
  print(' ' * i + '*' * (n - i))
```

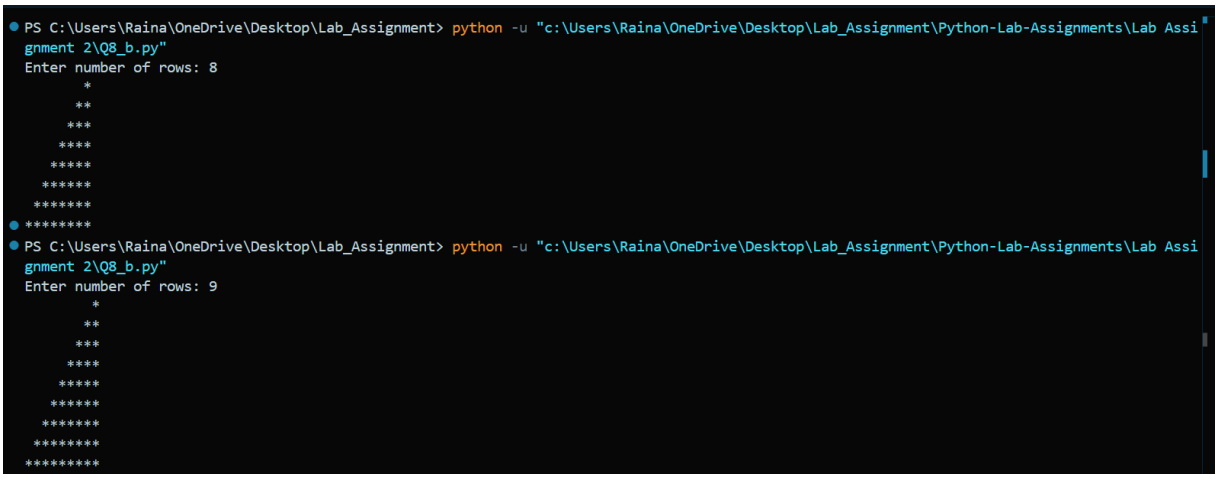
Output:



(b) Lower Right Triangle for character '*'

```
# input - take number of rows from user
n = int(input("Enter number of rows: "))

# output - print the pattern
for i in range(1, n + 1):
    print(' ' * (n - i) + '*' * i)
```

Output:



(C) Pyramid for any character entered through keyboard.

```
# input - take number of rows and the character from user
n = int(input("Enter number of rows: "))
char = input("Enter the character to use: ")

# output - print the pattern
for i in range(n):
    print(' ' * (n - 1 - i) + char * (2 * i + 1))
```

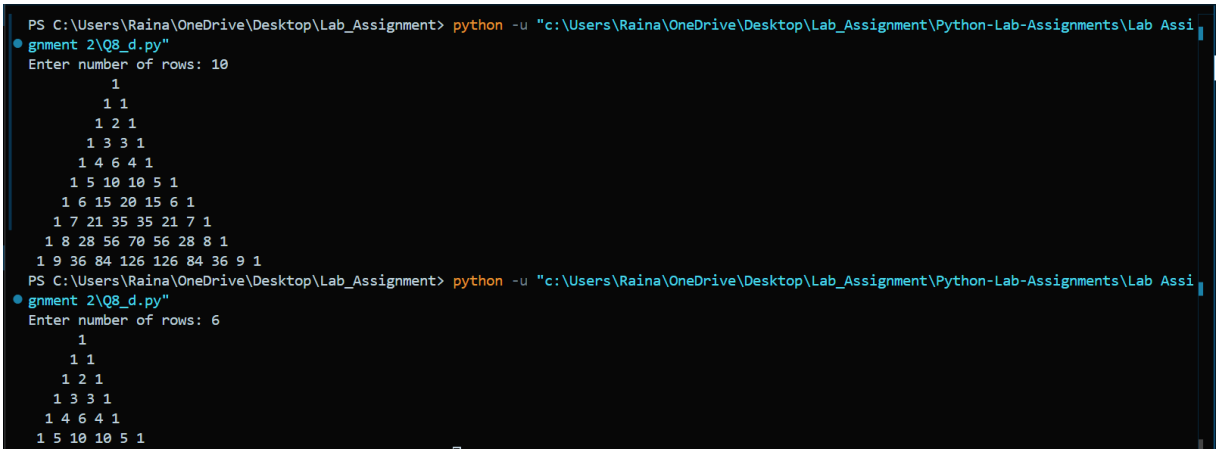
Output:

(d) Pascal Triangle

```
# input - take number of rows from user
n = int(input("Enter number of rows: "))
```

```
# output - print the pattern
for i in range(n):
    print(' ' * (n - i), end='')

    # The first number in any row is always 1
    num = 1
    for j in range(i + 1):
        print(num, end=' ')
        num = num * (i - j) // (j + 1)

    # Move to the next line after printing the row
    print()
```

Output:



Experiment-22

Objective: Program to find sum of series up to n terms (n is entered through console).

a. 1+3+5+7.....

b. 1 - 1/2 + 1/3 - 1/4 + 1/5 .....

c. 1! + 2! + 3! + 4! .....

d. 1^2 +2^ 2+3^ 2+.....

e. 1^3+2^3+3^3+.....

Code: (a) 1+3+5+7……

```
# input - take number of terms from user
n = int(input("Enter the number of terms: "))

total_sum = 0
term = 1

# Loop 'n' times, adding each term to the sum
for i in range(n):
    total_sum += term
    term += 2

# output - print the final sum
print(f"The sum of the series is: {total_sum}")
```

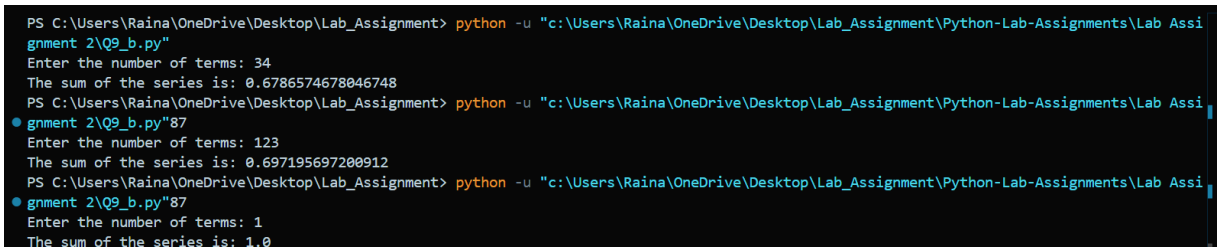
Output:

(b) 1 - 1/2 + 1/3 - 1/4 + 1/5 ……

```
# input - take number of terms from user
n = int(input("Enter the number of terms: "))

total_sum = 0.0

# Loop from 1 to n (inclusive) for each term
```

```
for i in range(1, n + 1):
    if i % 2 == 0:
        total_sum -= 1 / i
    else:
        total_sum += 1 / i

# output - print the final sum
print(f"The sum of the series is: {total_sum}")
```

Output:



(c) 1! + 2! + 3! + 4! .....

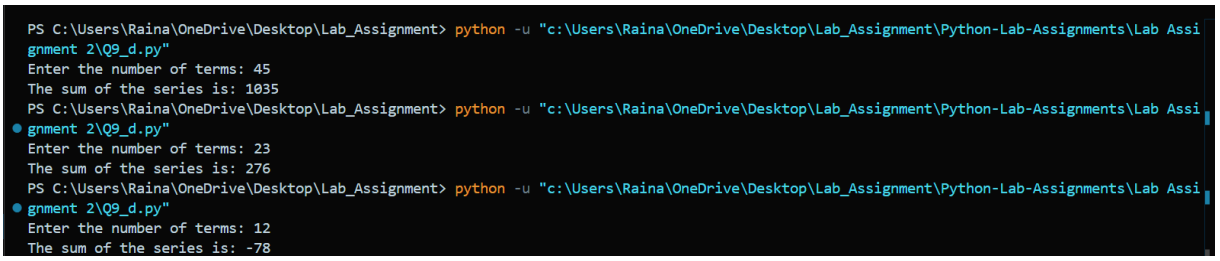
```
# input - take number of terms from user
n = int(input("Enter the number of terms: "))

total_sum = 0
factorial = 1

# Loop from 1 to n to calculate each term's factorial
for i in range(1, n + 1):
    # Calculate i! by multiplying the previous factorial (i-1)!
with i
    factorial *= i

    # Add the current factorial to the total sum
    total_sum += factorial

# output - print the final sum
print(f"The sum of the series is: {total_sum}")
```

Output:

(d) 1^2 +2^ 2+3^ 2+.....

```
# input - take number of terms from user
n = int(input("Enter the number of terms: "))
total_sum = 0
# Loop from 1 to n for each term in the series
for i in range(1, n + 1):
    # Calculate the value of the term, which is the square of the term number
    term = i ** 2
# Check if the term number is even or odd to apply the correct sign
    if i % 2 == 0:
        total_sum -= term
    else:
        total_sum += term

# output - print the final sum
print(f"The sum of the series is: {total_sum}")
```

Output:



(e) 1^3+2^3+3^3+.....

```
# input - take number of terms from user
n = int(input("Enter the number of terms: "))

total_sum = 0

# Loop from 1 to n for each term in the series
for i in range(1, n + 1):
    # Calculate the cube of the term number and add it to the sum
    total_sum += i ** 3

# output - print the final sum
print(f"The sum of the series is: {total_sum}")
```

Output:

```
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q9_e.py"
Enter the number of terms: 21
The sum of the series is: 53361
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q9_e.py"
Enter the number of terms: 100
The sum of the series is: 25502500
PS C:\Users\Raina\OneDrive\Desktop\Lab_Assignment> python -u "c:\Users\Raina\OneDrive\Desktop\Lab_Assignment\Python-Lab-Assignments\Lab Assignment 2\Q9_e.py"
Enter the number of terms: 312
The sum of the series is: 2384173584
```

LAB ASSIGNMENT 3

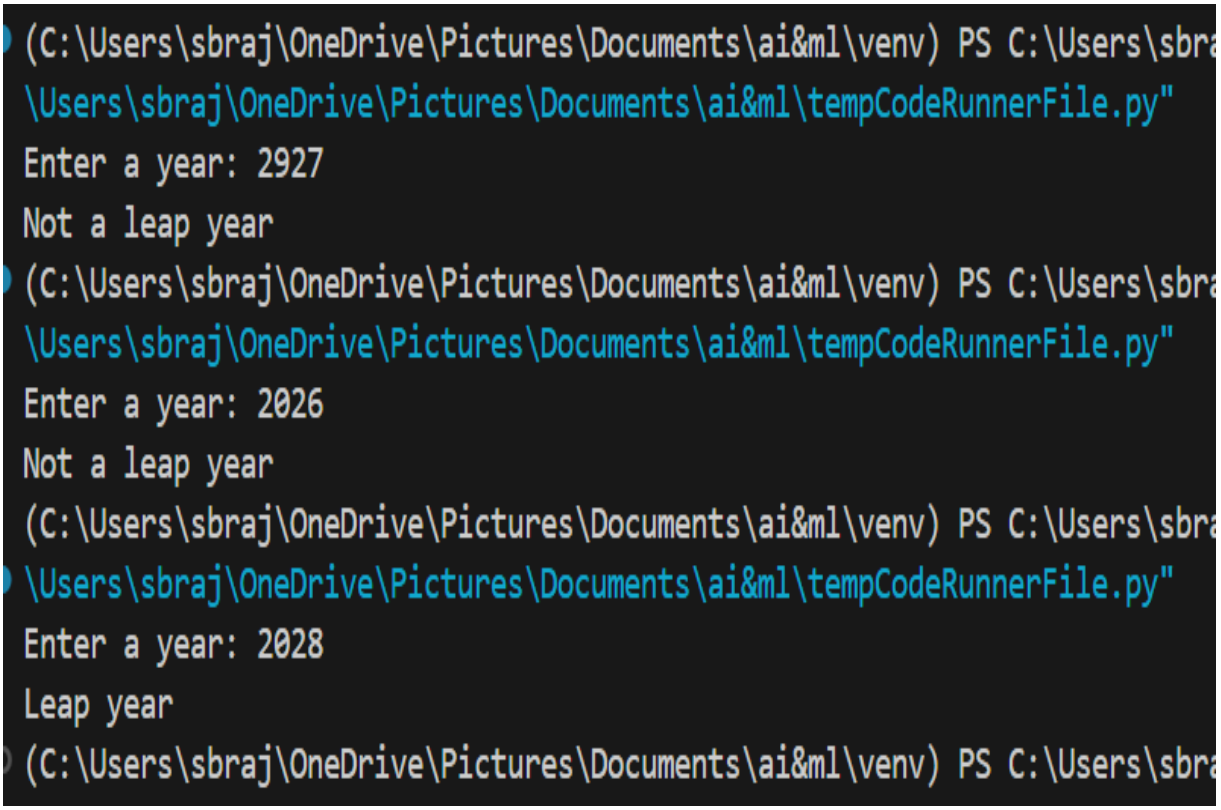
Experiment-23

Objective: Create a program to determine whether the number entered is a valid year and if so, whether it is a leap year.

Code 1:

```
year = int(input("Enter a year: "))


if year <= 0:
    print("Invalid year")
else:
    if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
        print("Leap year")
    else:
        print("Not a leap year")
```

Output:



Experiment-24

Objective: The librarian in a library wants an application that will calculate the due date for a book given the issue date. The no. of days in which the book is due can be decided by the librarian at the time of issuing a book. For e.g. If librarian enters the current date as 14- 01-99 and the no of days in which the book is due as 15, then your program should calculate the due date and give the output as 29-01-99.

Code 2:

```
d, m, y = map(int, input("Enter date (dd mm yy): ").split())
days = int(input("Enter no. of days: "))


# days in each month
month_days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]


# Leap year adjustment
if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
    month_days[1] = 29
d = d + days
while d > month_days[m-1]:
    d -= month_days[m-1]
    m += 1
    if m > 12:
        m = 1
        y += 1
        # recompute leap-year February
        if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
            month_days[1] = 29
        else:
            month_days[1] = 28
```

```
print("Due date:", d, m, y)
```

Output:

```
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documents\ai
_question1.py"
Enter date (dd mm yy): 22 09 2025
Enter no. of days: 10
Due date: 2 10 2025
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documents\ai
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documents\ai
ml\venv"
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\venv)
\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\a3_quest
Enter date (dd mm yy): 21 11 2006
Enter no. of days: 67
Due date: 27 1 2007
```

Experiment-25

Objective: Write a program that accepts input of a number of seconds, validates it and outputs the equivalent number of hours, minutes and seconds.

Code 3:

```
sec = int(input("Enter seconds: "))


if sec < 0:
    print("Invalid input")
else:
    hours = sec // 3600
    sec %= 3600
    minutes = sec // 60
    sec %= 60
    print("Hours:", hours, "Minutes:", minutes, "Seconds:", sec)
```

Output:

```
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documen
_question1.py"
Enter seconds: 34
Hours: 0 Minutes: 0 Seconds: 34
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documen
(base) PS C:\Users\sbraj\OneDrive\Pictures\Documen
ml\venv"
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\
\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\a3_
Enter seconds: 435
Hours: 0 Minutes: 7 Seconds: 15
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\
\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\a3_
Enter seconds: 65982
Hours: 18 Minutes: 19 Seconds: 42
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\
```

Experiment-26

Objective: Write a program which to print the multiplication table from 1 to m for n where m, n is entered by the user.

Code 4:

```
m = int(input("Enter m: "))
n = int(input("Enter n: "))


for i in range(1, n+1):
    for j in range(1, m+1):
        print(i, "x", j, "=", i*j)
    print()
```

Output:

```
(base) PS C:\Users\sbraj\OneDrive\
_question1.py"
Enter m: 3
Enter n: 5
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
```

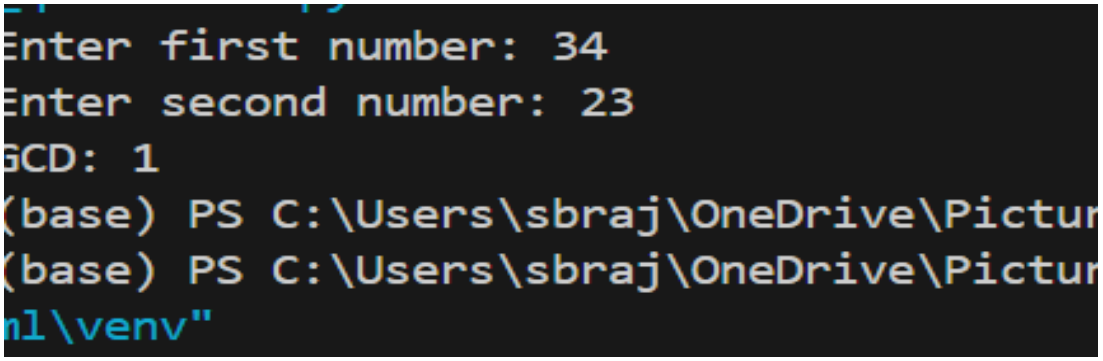
Experiment-27

Objective: Write a recursive function to calculate g.c.d of two numbers.

Code 5:

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)


a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("GCD:", gcd(a, b))
```

Output:



Experiment-28

Objective: Write a recursive function to calculate sum of n natural numbers. (n is entered through keyboard).

Code 6:

```
def sum_n(n):
    if n == 0:
        return 0
    return n + sum_n(n - 1)


n = int(input("Enter n: "))
print("Sum =", sum_n(n))
```

Output:

```
\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\a3_question1.
Enter n: 67
Sum = 2278
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\venv) PS C
43
(C:\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\venv) PS C
\Users\sbraj\OneDrive\Pictures\Documents\ai&ml\a3_question1.
Enter n: 32
Sum = 528
```

Experiment-29

Objective: Write a recursive function to calculate factorial of a number.

Code 7:

```
def fact(n):
    if n == 0 or n == 1:
        return 1
    return n * fact(n - 1)

n = int(input("Enter number: "))
print("Factorial =", fact(n))
```

Output:

```
(C:\Users\sbraj\OneDrive\Pictures\Documents\
\Users\sbraj\OneDrive\Pictures\Documents\ai&
Enter number: 12
Factorial = 479001600
```

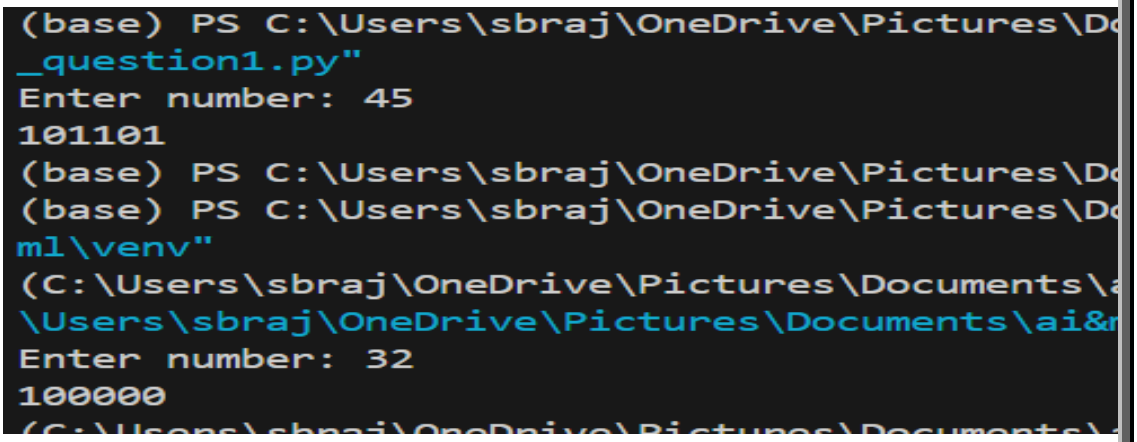
Experiment-30

Objective: Write a recursive function to print binary representation of integer n (entered through console).

Code 8:

```
def to_binary(n):
    if n > 1:
        to_binary(n // 2)
    print(n % 2, end='')


n = int(input("Enter number: "))
to_binary(n)
```

Output:



Experiment-31

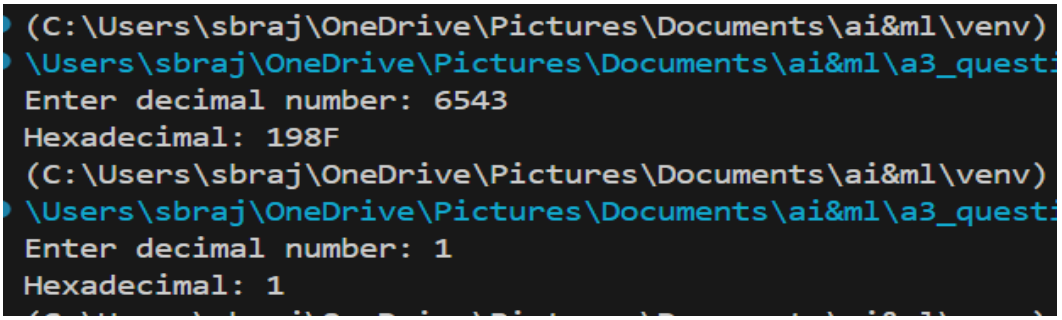
Objective: Write a function to convert a decimal number to its hexadecimal equivalent and check whether this function can be converted to recursive function. If yes, then change code and save it in new a file.

Code 9:

```
num = int(input("Enter decimal number: "))
hex_digits = "0123456789ABCDEF"
result = ""

if num == 0:
    result = "0"
```

```
else:
    while num > 0:
        result = hex_digits[num % 16] + result
        num //= 16

print("Hexadecimal:", result)
```

Output:



Experiment-32

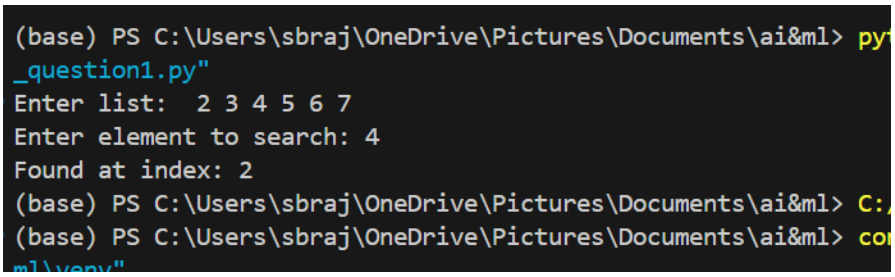
Objective: Write a recursive function to search an element in the given list using binary search technique.

Code 10:

```
def binary_search(arr, left, right, x):
    if left > right:
        return -1
    mid = (left + right) // 2

    if arr[mid] == x:
        return mid
    elif x < arr[mid]:
        return binary_search(arr, left, mid - 1, x)
    else:
        return binary_search(arr, mid + 1, right, x)
```

```
arr = sorted(list(map(int, input("Enter list: ").split())))
x = int(input("Enter element to search: "))

index = binary_search(arr, 0, len(arr)-1, x)

print("Found at index:", index if index != -1 else "Not found")
```

Output:



LAB ASSIGNMENT 4

Experiment-33

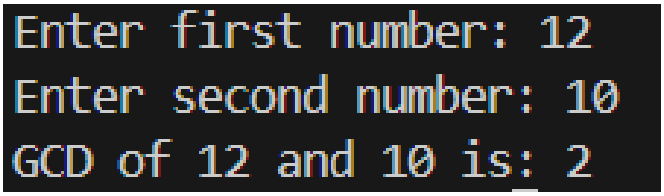
Objective: Create a program to compute g.c.d. of two numbers using Euclidean algorithm.

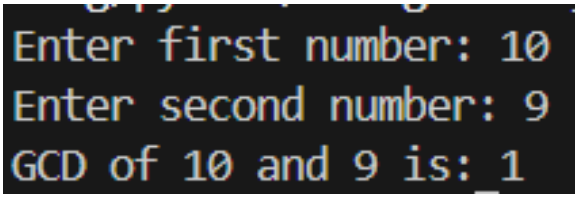
Code 1:

```
def gcd(a, b):
    while b != 0:
        temp = b
        b = a % b
        a = temp
    return a
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a > 0 and b > 0:
    result = gcd(a, b)
```

```
    print(f"GCD of {a} and {b} is: {result}")
else:
    print("Please enter positive numbers")
```

Output:





Experiment-34:

Objective: Create a program to find the sum of numbers having odd index and multiplication of numbers having even index in one dimensional array.

Code 2:

```
n = int(input("Enter size of array: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i}: "))
    arr.append(element)
sum_odd_index = 0
product_even_index = 1
for i in range(len(arr)):
    if i % 2 != 0:  # odd index
        sum_odd_index += arr[i]
    else:  # even index
        product_even_index *= arr[i]
print(f"Sum of elements at odd indices: {sum_odd_index}")
print(f"Product of elements at even indices: {product_even_index}")
```

Output:

```
Enter size of array: 5
Enter element 0: 2
Enter element 1: 4
Enter element 2: 3
Enter element 3: 1
Enter element 4: 5
Sum of elements at odd indices: 5
Product of elements at even indices: 30
```

```
Enter size of array: 3
Enter element 0: -5
Enter element 1: -7
Enter element 2: 0
Sum of elements at odd indices: -7
Product of elements at even indices: 0
```

Experiment-35:

Objective: Write a program to print numbers divisible by 3 and 7 between 1 to 500.

Code :

```
print("Numbers divisible by both 3 and 7 between 1 to 500:")
for num in range(1, 501):
    if num % 3 == 0 and num % 7 == 0:
        print(num, end=" ")
print()
```

Output:

```
Numbers divisible by both 3 and 7 between 1 to 500:
21 42 63 84 105 126 147 168 189 210 231 252 273 294 315 336 357 378 399 420 441 462 483
```

Experiment-36:

Objective: Create a program to read one dimensional array from the keyboard and apply the following operations on it:

a. To search an element entered through console.

b. To display the reverse list with and without using second array.

c. To sort the list with and without using second array.

d. To find the mean of the list.

Code 4:

```
import statistics
def search_element(arr, element):
    if element in arr:return arr.index(element);return -1
def reverse_with_array(arr):
    return arr[::-1]
def reverse_without_array(arr):
    reversed_arr = []
    for i in range(len(arr)-1, -1, -1): reversed_arr.append(arr[i])
    return reversed_arr
def sort_with_array(arr):
    return sorted(arr)
def sort_without_array(arr):
    arr_copy = arr[:]
    for i in range(len(arr_copy)):
        for j in range(len(arr_copy)-1-i):
            if arr_copy[j] > arr_copy[j+1]: arr_copy[j],
arr_copy[j+1] = arr_copy[j+1], arr_copy[j];return arr_copy
def find_mean(arr): return statistics.mean(arr)
n = int(input("Enter size of array: "));arr = []
for i in range(n): element = int(input(f"Enter element {i}: "))
    arr.append(element)
print("\n=== Array Operations ===")
print(f"Original array: {arr}")
# Search
search_val = int(input("\nEnter element to search: "))
```

```
index = search_element(arr, search_val)

if index != -1:

    print(f"Element found at index: {index}")

else:

    print("Element not found")

# Reverse

print(f"\nReverse (using slicing): {reverse_with_array(arr)}")

print(f"Reverse (without using second array):
{reverse_without_array(arr)}")

# Sort

print(f"\nSorted (using sorted()): {sort_with_array(arr)}")

print(f"Sorted (without using second array):
{sort_without_array(arr)}")

# Mean

print(f"\nMean: {find_mean(arr)}")
```

Output:

```
Enter size of array: 3
Enter element 0: 4
Enter element 1: -5
Enter element 2: 0

=== Array Operations ===
Original array: [4, -5, 0]

Enter element to search: 6
Element not found
```

```
Reverse (using slicing): [0, -5, 4]
Reverse (without using second array): [0, -5, 4]

Sorted (using sorted()): [-5, 0, 4]
Sorted (without using second array): [-5, 0, 4]
```

Experiment-37: Create a program to read one dimensional character type array and display all the vowels present in it with their indices.

Code 5:

```
n = int(input("Enter size of character array: "))
char_arr = []
for i in range(n):
    char = input(f"Enter character {i}: ").strip()
    if len(char) == 1:
        char_arr.append(char)
    else:
        print("Please enter single character only")
        char = input(f"Enter character {i}: ").strip()
        char_arr.append(char)
vowels = ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
print("\nVowels present in array with indices:");found = False
for idx, char in enumerate(char_arr):
    if char in vowels: print(f"'{char}' at index {idx}")
        found = True
if not found:print("No vowels found")
```

Output:

```
Enter size of character array: 4
Enter character 0: a
Enter character 1: r
Enter character 2: q
Enter character 3: l

Vowels present in array with indices:
'a' at index 0
```

```
Enter size of character array: 3
Enter character 0: r
Enter character 1: q
Enter character 2: s

Vowels present in array with indices:
No vowels found
```

Experiment-38: Create a program to read one dimensional character type array and reprint after changing the case of each character.

Code 6:

```
n = int(input("Enter size of character array: "))
char_arr = []
for i in range(n): char = input(f"Enter character {i}: ").strip()
    if len(char) == 1: char_arr.append(char)
    else: print("Please enter single character only")
print("\nOriginal array:")
print(char_arr)
print("\nArray after changing case:")
for char in char_arr: if char.isupper():print(char.lower(), end=" ")
    elif char.islower():print(char.upper(), end=" ")
    else: print(char, end=" ");print()
```

Output

```
Enter size of character array: 4
Enter character 0: r
Enter character 1: A
Enter character 2: t
Enter character 3: E

Original array:
['r', 'A', 't', 'E']

Array after changing case:
R a T e
```

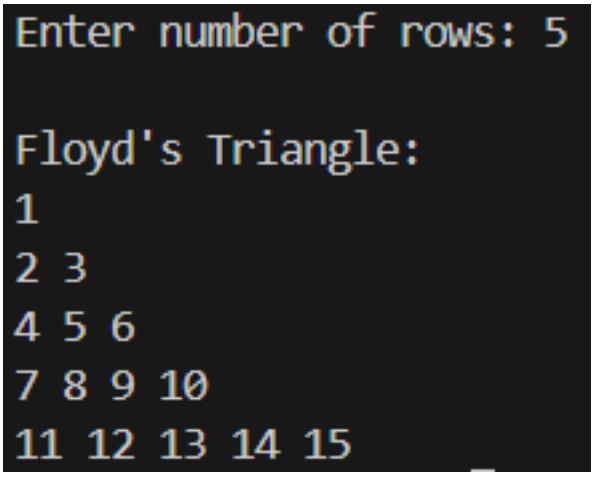
```
Enter size of character array: 1
Enter character 0: jey
Please enter single character only

Original array:
[]

Array after changing case:
```

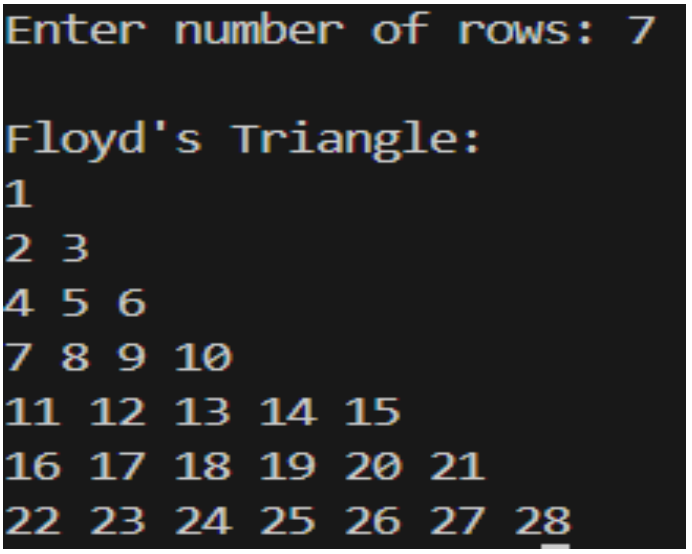
Experiment-39: Create a program to print Floyd's triangle for n number of rows.

Code :

```
n = int(input("Enter number of rows: "))
num = 1

print("\nFloyd's Triangle:")
for i in range(1, n+1):
    for j in range(i):
        print(num, end=" ")
        num += 1
    print()
```

Output:





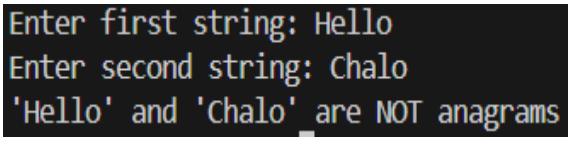
Experiment-40: Create a program to check whether two strings are anagrams or not.

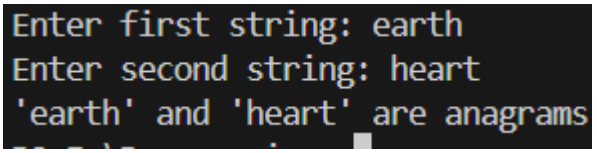
Code 8:

```
def are_anagrams(str1, str2):
    str1 = str1.replace(" ", "").lower()
```

```
    str2 = str2.replace(" ", "").lower()

    return sorted(str1) == sorted(str2)

string1 = input("Enter first string: ")
string2 = input("Enter second string: ")

if are_anagrams(string1, string2):
    print(f"'{string1}' and '{string2}' are anagrams")
else:
    print(f"'{string1}' and '{string2}' are NOT anagrams")
```

Output:





Experiment-41:

Write a program to perform the following operations on one-dimensional array (entered through console).

a. Delete an element from the list (if found).

b. Delete an element from the specified index.

c. Insert an element at specified index.

d. Insert an element in ascending/descending order.

Code 9:

```
n = int(input("Enter size of array: "))
```

```
arr = []
for i in range(n):
    element = int(input(f"Enter element {i}: "))
    arr.append(element)
print(f"Original array: {arr}")
while True:
    print("\n=== Array Operations ===")
    print("1. Delete element by value")
    print("2. Delete element by index")
    print("3. Insert element at index")
    print("4. Insert element in sorted order")
    print("5. Exit")
    choice = input("Enter choice (1-5): ")
    if choice == '1':
        val = int(input("Enter value to delete: "))
        if val in arr: arr.remove(val)
            print(f"Array after deletion: {arr}")
        else: print("Element not found")
    elif choice == '2': idx = int(input("Enter index to delete: "))
        if 0 <= idx < len(arr):
            arr.pop(idx)
            print(f"Array after deletion: {arr}")
        else: print("Invalid index")
    elif choice == '3': idx = int(input("Enter index to insert: "))
        val = int(input("Enter value to insert: "))
        if 0 <= idx <= len(arr):arr.insert(idx, val)
            print(f"Array after insertion: {arr}")
        else: print("Invalid index")
```

```
    elif choice == '4': val = int(input("Enter value to insert: "))
        arr.append(val)
        arr.sort()
        print(f"Array after sorted insertion: {arr}")
    elif choice == '5': break
    else: print("Invalid choice")
```

Output:

```
=== Array Operations ===
1. Delete element by value
2. Delete element by index
3. Insert element at index
4. Insert element in sorted order
5. Exit
Enter choice (1-5): 4
Enter value to insert: 54
Array after sorted insertion: [1, 3, 7, 54]

=== Array Operations ===
1. Delete element by value
2. Delete element by index
3. Insert element at index
4. Insert element in sorted order
5. Exit
Enter choice (1-5): 5
```

```
Enter size of array: 4
Enter element 0: 1
Enter element 1: 4
Enter element 2: 7
Enter element 3: 3
Original array: [1, 4, 7, 3]

=== Array Operations ===
1. Delete element by value
2. Delete element by index
3. Insert element at index
4. Insert element in sorted order
5. Exit
Enter choice (1-5): 1
Enter value to delete: 4
Array after deletion: [1, 7, 3]
```

Experiment-42: Write a program to perform the following operations:

a. Transpose of matrix.

b. Addition of two matrices (if possible)

c. Subtraction of two matrices (if possible).

d. Multiplication of two matrices (if possible).

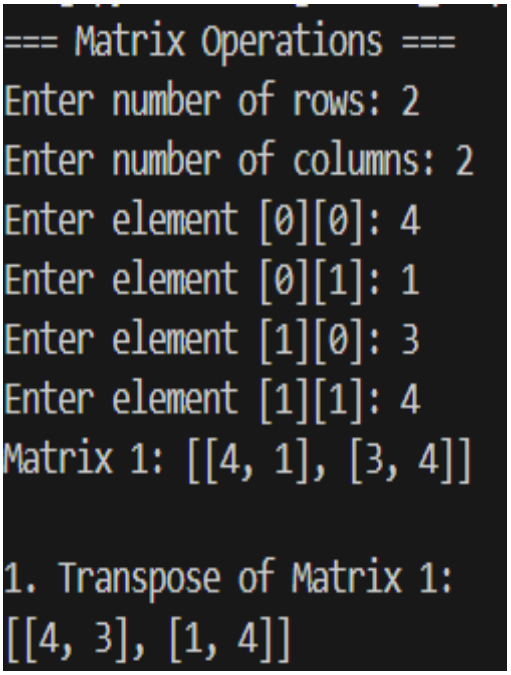
Code 10:

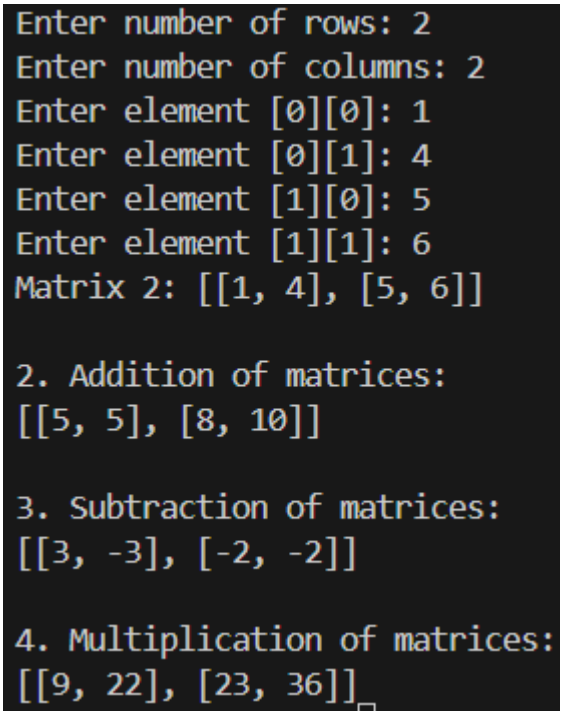
```
def transpose(matrix):
```

```
    return [[matrix[j][i] for j in range(len(matrix))] for i in
range(len(matrix[0]))]

def add_matrices(mat1, mat2):

    if len(mat1) == len(mat2) and len(mat1[0]) == len(mat2[0]):
return [[mat1[i][j] + mat2[i][j] for j in range(len(mat1[0]))] for i
in range(len(mat1))]

    return None

def subtract_matrices(mat1, mat2):

    if len(mat1) == len(mat2) and len(mat1[0]) ==
len(mat2[0]):return [[mat1[i][j] - mat2[i][j] for j in
range(len(mat1[0]))] for i in range(len(mat1))];    return None

def multiply_matrices(mat1, mat2):

    if len(mat1[0]) == len(mat2):result = [[0 for _ in
range(len(mat2[0]))] for _ in range(len(mat1))]

        for i in range(len(mat1)):

            for j in range(len(mat2[0])):

                for k in range(len(mat2)):

                    result[i][j] += mat1[i][k] * mat2[k][j];return
result;return None

def read_matrix():

    rows = int(input("Enter number of rows: "))

    cols = int(input("Enter number of columns: "))

    matrix = []

    for i in range(rows):row = [];for j in range(cols): element =
int(input(f"Enter element [{i}][{j}]: "));row.append(element);
matrix.append(row); return matrix

print("=== Matrix Operations ===")

matrix1 = read_matrix()

print(f"Matrix 1: {matrix1}")

print("\n1. Transpose of Matrix 1:")

print(transpose(matrix1))

matrix2 = read_matrix()
```

```
print(f"Matrix 2: {matrix2}")
print("\n2. Addition of matrices:")
result = add_matrices(matrix1, matrix2)
if result:print(result)
else:print("Cannot add: dimensions not compatible")
print("\n3. Subtraction of matrices:")
result = subtract_matrices(matrix1, matrix2)
if result:print(result)
else:print("Cannot subtract: dimensions not compatible")
print("\n4. Multiplication of matrices:")
result = multiply_matrices(matrix1, matrix2)
if result:print(result)
else:print("Cannot multiply: dimensions not compatible")
```

Output:





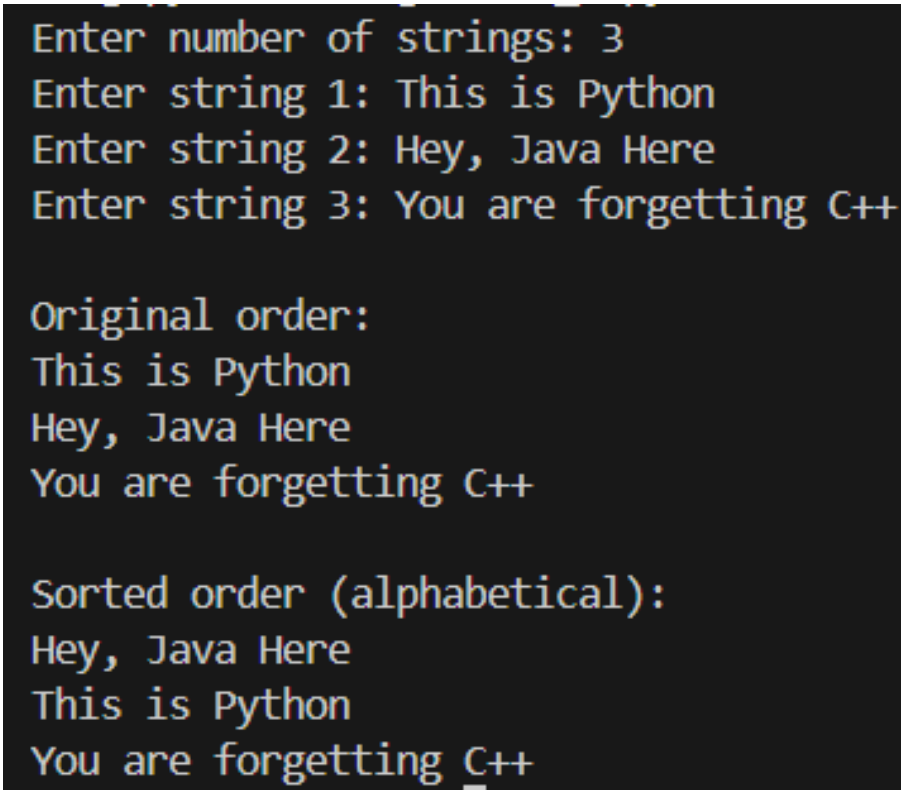
LAB ASSIGNMENT-5

Experiment-43: Create a program to sort a list of strings into alphabetical order (as in dictionary).

Code 1:

```
strings = [];n = int(input("Enter number of strings: "))
for i in range(n):s = input(f"Enter string {i+1}: ")
    strings.append(s)
sorted_strings = sorted(strings)
print("\nOriginal order:")
for s in strings:print(s)
print("\nSorted order (alphabetical):")
for s in sorted_strings: print(s)
```

Output:



Experiment-44: Write a program that analysis the line of text to check the following:

a. Number of characters.

b. Digits.

c. White spaces.

d. Vowels.

e. Consonants.

f. Special Characters.

Code 2:

```
text = input("Enter a line of text: ")

total_chars = len(text)

digits = sum(1 for c in text if c.isdigit())

whitespaces = sum(1 for c in text if c.isspace())

vowels = sum(1 for c in text.lower() if c in 'aeiou')

consonants = sum(1 for c in text if c.isalpha() and c.lower() not in
'aeiou')

special_chars = sum(1 for c in text if not c.isalnum() and not
c.isspace())

print(f"Number of characters: {total_chars}")

print(f"Number of digits: {digits}")

print(f"Number of white spaces: {whitespaces}")

print(f"Number of vowels: {vowels}")

print(f"Number of consonants: {consonants}")

print(f"Number of special characters: {special_chars}")
```

Output:

```
Enter a line of text: Quick Brown Fox Jumps over Lazy dOg
Number of characters: 35
Number of digits: 0
Number of white spaces: 6
Number of vowels: 9
Number of consonants: 20
Number of special characters: 0
```

Experiment-45:

Write a menu driven program to convert a number entered through console into words. E.g. 5312 should be converted in both ways like FIVE THREE ONE TWO or Five

Thousand Three Hundred Twelve depending upon choice entered by the user.

Code 3:

```
ones = ["", "One", "Two", "Three", "Four", "Five", "Six", "Seven",
"Eight", "Nine"]

teens = ["Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
"Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"]

tens = ["", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty",
"Seventy", "Eighty", "Ninety"]

scales = ["", "Thousand", "Million", "Billion"]

digit_words =
["Zero","One","Two","Three","Four","Five","Six","Seven","Eight",
"Nine"]

def number_to_words(num):

    if num == 0:

        return "Zero"

    groups = []
```

```
    while num > 0:
        groups.append(num % 1000)
        num //= 1000
    result = []
    for i in range(len(groups) - 1, -1, -1):
        if groups[i] == 0:
            continue
        group_words = convert_group(groups[i])
        if i > 0: group_words += " " + scales[i]
        result.append(group_words)
    return " ".join(result)
def convert_group(num):
    result = []

    hundreds = num // 100
    if hundreds > 0:
        result.append(ones[hundreds] + " Hundred")
    remainder = num % 100
    if remainder >= 20:
        tens_digit = remainder // 10
        ones_digit = remainder % 10
        result.append(tens[tens_digit] + (" " + ones[ones_digit] if
ones_digit > 0 else ""))
    elif remainder >= 10:
        result.append(teens[remainder - 10])
    elif remainder > 0:
        result.append(ones[remainder])

    return " ".join(result)
```

```
def number_to_individual_words(num_str):
    return " ".join(digit_words[int(d)] for d in num_str)


while True:
    print("\n1. Convert to words (e.g., 5312 = Five Thousand Three Hundred Twelve)")
    print("2. Convert to individual digit words (e.g., 5312 = Five Three One Two)")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        num = int(input("Enter a number: "))
        print(f"{num} = {number_to_words(num)}")

    elif choice == '2':
        num_str = input("Enter a number: ")
        print(f"{num_str} = {number_to_individual_words(num_str)}")

    elif choice == '3':
        break

    else:
        print("Invalid choice")
```

Output:

```
1. Convert to words (e.g., 5312 = Five Thousand Three Hundred Twelve)
2. Convert to individual digit words (e.g., 5312 = Five Three One Two)
3. Exit
Enter choice: 1
Enter a number: 54123
54123 = Fifty Four Thousand One Hundred Twenty Three
```

```
1. Convert to words (e.g., 5312 = Five Thousand Three Hundred Twelve)
2. Convert to individual digit words (e.g., 5312 = Five Three One Two)
3. Exit
Enter choice: 2
Enter a number: 5789412
5789412 = Five Seven Eight Nine Four One Two
```

Experiment-46:

Write a program to check the validity of the string entered through console as:

a. Email ID

b. Date (entered in any valid format)

Code 4:

```
import re

from datetime import datetime

def is_valid_email(email):

    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    return re.match(pattern, email) is not None

def is_valid_date(date_str):

    date_formats = ['%d/%m/%Y', '%d-%m-%Y', '%d.%m.%Y', '%m/%d/%Y',
'%m-%d-%Y', '%m.%d.%Y', '%Y/%m/%d', '%Y-%m-%d', '%Y.%m.%d',
'%d/%m/%y', '%d-%m-%y', '%d.%m.%y',%B %d, %Y', '%b %d, %Y', '%d %B
%Y', '%d %b %Y' ]

    for fmt in date_formats:

        try:
```

```
            datetime.strptime(date_str, fmt)
            return True
        except ValueError:
            continue
    return False
while True:
    print("\n1. Validate Email ID")
    print("2. Validate Date")
    print("3. Exit")
    choice = input("Enter choice: ")
    if choice == '1':
        email = input("Enter email ID: ")
        if is_valid_email(email): print("Valid Email ID")
        else: print("Invalid Email ID")
    elif choice == '2': date = input("Enter date: ")
        if is_valid_date(date): print("Valid Date")
        else: print("Invalid Date")
    elif choice == '3': break
    else: print("Invalid choice")
```

Output:

```
1. Validate Email ID
2. Validate Date
3. Exit
Enter choice: 1
Enter email ID: efnonfwpe#k;frmv
Invalid Email ID

1. Validate Email ID
2. Validate Date
3. Exit
Enter choice: 2
Enter date: 903473
Invalid Date
```

```
1. Validate Email ID
2. Validate Date
3. Exit
Enter choice: 1
Enter email ID: 24218@iiitu.ac.i
Valid Email ID

1. Validate Email ID
2. Validate Date
3. Exit
Enter choice: 2
Enter date: 1453
Invalid Date
```

Experiment-47: Write a menu driven program using your own functions to perform following operations on string/s entered through console

a. Reverse a string.

b. Calculate length of string.

c. Change case.

d. Change to Upper Case.

e. Change to Lower Case.

f. Append two strings.

g. Sort the characters of string.

Code 5:

```
def reverse_string(s):
    return s[::-1]
def string_length(s):
    return len(s)
def change_case(s):
    result = ""
    for char in s:
        if char.isupper():
            result += char.lower()
        elif char.islower():
            result += char.upper()
        else:
            result += char
    return result
def to_upper(s):
    return s.upper()
```

```
def to_lower(s):
    return s.lower()
def append_strings(s1, s2):
    return s1 + s2
def sort_characters(s):
    return "".join(sorted(s))
while True:
    print("\n=== String Operations ===")
    print("1. Reverse a string")
    print("2. Calculate length of string")
    print("3. Change case")
    print("4. Change to Upper Case")
    print("5. Change to Lower Case")
    print("6. Append strings")
    print("7. Sort characters of string")
    print("8. Exit")
    choice = input("Enter choice: ")
    if choice == '1':
        s = input("Enter string: ")
        print(f"Reversed: {reverse_string(s)}")
    elif choice == '2':
        s = input("Enter string: ")
        print(f"Length: {string_length(s)}")
    elif choice == '3':
        s = input("Enter string: ")
        print(f"Case changed: {change_case(s)}")
    elif choice == '4':
        s = input("Enter string: ")
```

```
        print(f"Upper case: {to_upper(s)}")
    elif choice == '5':
        s = input("Enter string: ")
        print(f"Lower case: {to_lower(s)}")
    elif choice == '6':
        s1 = input("Enter first string: ")
        s2 = input("Enter second string: ")
        print(f"Appended: {append_strings(s1, s2)}")
    elif choice == '7':
        s = input("Enter string: ")
        print(f"Sorted: {sort_characters(s)}")
    elif choice == '8':
        break
    else:
        print("Invalid choice")
```

Output:

```
=== String Operations ===
1. Reverse a string
2. Calculate length of string
3. Change case
4. Change to Upper Case
5. Change to Lower Case
6. Append strings
7. Sort characters of string
8. Exit
Enter choice: 6
Enter first string: Hello
Enter second string: How are you
Appended: HelloHow are you
```

```
=== String Operations ===
1. Reverse a string
2. Calculate length of string
3. Change case
4. Change to Upper Case
5. Change to Lower Case
6. Append strings
7. Sort characters of string
8. Exit
Enter choice: 3
Enter string: HelLo HoW arE yOu
Case changed: hELlO hOw ARe YoU
```

LAB ASSIGNMENT 6

Experiment-48: “Story.txt” file contains the following story. Read the content and display it on the screen using a program:

Code 1:

```
with open("Story.txt", "r") as file: content =
file.read()print(content)
```

Output:

```
The Dean thought for a minute and said they can have the re-test after 3 days. They
thanked him and said they will be ready by that time. On the third day, they appeared
before the Dean. The Dean said that as this was a Special Condition Test, all four were
required to sit in separate classrooms for the test. They all agreed as they had prepared
well in the last 3 days. The Test consisted of only 2 questions with the total of 100
Points:
1) Your Name? __________ (1 Points)
2) Which tire burst? __________ (99 Points)
Options â€“ (a) Front Left (b) Front Right (c) Back Left (d) Back Right
```

Program 2: Read the file Story.txt (in Q1) and display the contents on screen line by line using program.

Code 2:

```
with open("Story.txt", "r") as file: for line in file: print(line,
end="")
```

Output:

```
The Dean thought for a minute and said they can have the re-test after 3 days. They
thanked him and said they will be ready by that time. On the third day, they appeared
before the Dean. The Dean said that as this was a Special Condition Test, all four were
required to sit in separate classrooms for the test. They all agreed as they had prepared
well in the last 3 days. The Test consisted of only 2 questions with the total of 100
Points:
1) Your Name? __________ (1 Points)
2) Which tire burst? __________ (99 Points)
Options â€“ (a) Front Left (b) Front Right (c) Back Left (d) Back Right
```

Experiment-49: Write a program to count:

a) Number of characters in a file

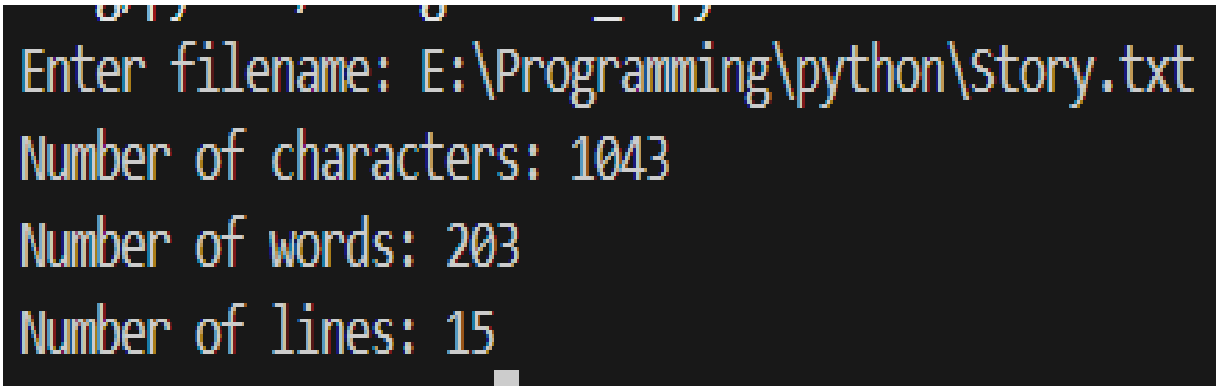
b) Number of words in a file

c) Number of lines in a file

Code 3:

```
filename = input("Enter filename: ")

with open(filename, "r") as file:
    content = file.read()
    characters = len(content)
    words = len(content.split())
    lines = len(content.split('\n'))

    print(f"Number of characters: {characters}")
    print(f"Number of words: {words}")
    print(f"Number of lines: {lines}")
```

Output:



Experiment-50: How would you copy the contents of a file into another file using a program?

Code 4:

```
source = input("Enter source filename: ")
destination = input("Enter destination filename: ")

with open(source, "r") as src:
    content = src.read()

with open(destination, "w") as dest:
    dest.write(content)

print(f"File copied from {source} to {destination}")
```

Output:

```
Enter source filename: E:\Programming\python\Story.txt
Enter destination filename: lab.txt
File copied from E:\Programming\python\Story.txt to lab.txt
```

lab.txt

```
One night four college students were out partying late at night and didn't study for the
test which was scheduled for the next day. In the morning, they thought of a plan. They
made themselves look dirty with grease and dirt. Then they went to the Dean and said
they had gone out to a wedding last night and on their way back the tire of their car
burst and they had to push the car all the way back. So they were in no condition to take
the test.
The Dean thought for a minute and said they can have the re-test after 3 days. They
thanked him and said they will be ready by that time. On the third day, they appeared
before the Dean. The Dean said that as this was a Special Condition Test, all four were
required to sit in separate classrooms for the test. They all agreed as they had prepared
well in the last 3 days. The Test consisted of only 2 questions with the total of 100
Points:
1) Your Name? _________ (1 Points)
2) Which tire burst? _________ (99 Points)
Options - (a) Front Left (b) Front Right (c) Back Left (d) Back Right
```

Experiment-51: Read marks500.txt to find min and max of the marks. This file can be found at on the lab assignment page:

Code 5:

```
import pandas as pd

df = pd.read_csv("E:\Programming\marks500.txt", sep=r"\s+",
header=None, names=["roll", "marks"]); df = df[df["roll"] != -9999]

print("Minimum marks:", df["marks"].min());print("Maximum marks:",
df["marks"].max())
```

Output:

```
Minimum marks: 0.0
Maximum marks: 100.0
```

Program 6: Read marks500.txt. Which roll number is the topper and print the roll numbers who failed (marks below 33)

Code 6:

```
import pandas as pd

df =
pd.read_csv("marks500.txt",sep=r"\s+",header=None,names=["roll",
"marks"],engine="python");df = df[df["roll"] != -9999];topper_row =
df.loc[df["marks"].idxmax()];print("Topper roll number:",
int(topper_row["roll"]));print("Topper marks:",
int(topper_row["marks"]))

failed = df[df["marks"] < 33]["roll"];print("\nRoll numbers who
failed (marks < 33):")

for r in failed:
print(int(r))
```

Output:

```
In [4]: %runfile C:/Users/Avadh/.spyder-py3/untitled2.py
Topper roll number: 1450
Topper marks: 100

Roll numbers who failed (marks < 33):
1004
1007
1009
1010
1012
1013
1014
1016
```

Experiment-52: Read marks500.txt. Find average and standard deviation of these 500 marks and save this into a new file MarksStats.txt

Code 7:

```
import pandas as pd

df=pd.read_csv("marks500.txt",sep=r"\s+",header=None,names=["roll","
marks"],engine="python")

df = df[df["roll"] != -9999]

average = df["marks"].mean()

std_dev = df["marks"].std()   # sample standard deviation (N-1)

with open("MarksStats.txt", "w") as f:

    f.write(f"Average: {average}\n")

    f.write(f"Standard Deviation: {std_dev}\n")

print("Average:", average)

print("Standard Deviation:", std_dev)

print("Stats saved to MarksStats.txt")
```

Output:

```
Average: 51.246
Standard Deviation: 28.941124
Stats saved to MarksStats.txt
```

LAB ASSIGNMENT 7

Experiment-53: WAP to read two times (Hours, Minutes, Seconds) and add them.

a. using member function

Code 1:

```
class Time:

    def __init__(self, h=0, m=0, s=0):

        self.hours = h;  self.minutes = m; self.seconds = s

    def addTime(self, other):

        total_sec = (self.seconds + other.seconds) % 60carry_min =
(self.seconds + other.seconds) // 60;total_min = (self.minutes +
other.minutes + carry_min) % 60;

        carry_hr = (self.minutes + other.minutes + carry_min) //
60;        total_hr = self.hours + other.hours +
carry_hr;        return Time(total_hr, total_min, total_sec)

    def display(self):

        print(f"{self.hours:02d}:{self.minutes:02d}:{self.seconds:02
d}")

t1 = Time(int(input("Enter hours 1: ")), int(input("Enter minutes 1:
")), int(input("Enter seconds 1: ")))

t2 = Time(int(input("Enter hours 2: ")), int(input("Enter minutes 2:
")), int(input("Enter seconds 2: ")))

print("\nUsing member function:");result1 =
t1.addTime(t2);result1.display()
```

Output:

```
Enter hours 1: 12
Enter minutes 1: 14
Enter seconds 1: 56
Enter hours 2: 11
Enter minutes 2: 59
Enter seconds 2: 14

Using member function:
24:14:10
```

```
Enter hours 1: 7
Enter minutes 1: 5
Enter seconds 1: 14
Enter hours 2: 2
Enter minutes 2: 53
Enter seconds 2: 45

Using member function:
09:58:59
```

Experiment-54: WAP to design a class cylinder and the operations on cylinder as follows:

a. Calculate the volume

b. Calculate the surface area

c. Calculate the area of cylinder base

d. Set the height, radius and center of the base.

Code 2:

```
import math
class Cylinder:
    def __init__(self, r=0, h=0, cx=0, cy=0):
        self.radius = r
        self.height = h
        self.center_x = cx
        self.center_y = cy
    def setProperties(self, r, h, cx, cy):
        self.radius = r
        self.height = h
        self.center_x = cx
        self.center_y = cy
    def volume(self):
        return math.pi * self.radius ** 2 * self.height
    def surfaceArea(self):
        return 2 * math.pi * self.radius * self.height + 2 * math.pi
* self.radius ** 2
    def baseArea(self):
        return math.pi * self.radius ** 2
    def display(self):
```

```
        print(f"Radius: {self.radius}, Height: {self.height}")
        print(f"Center: ({self.center_x}, {self.center_y})")
        print(f"Volume: {self.volume():.2f}")
        print(f"Surface Area: {self.surfaceArea():.2f}")
        print(f"Base Area: {self.baseArea():.2f}")
cyl = Cylinder()
cyl.setProperties(float(input("Enter radius: ")), float(input("Enter height: ")), float(input("Enter center x: ")), float(input("Enter center y: ")))
cyl.display()
```

Output:

```
Enter radius: 4
Enter height: 2
Enter center x: 8
Enter center y: 0
Radius: 4.0, Height: 2.0
Center: (8.0, 0.0)
Volume: 100.53
Surface Area: 150.80
Base Area: 50.27
```

```
Enter radius: 86
Enter height: 4
Enter center x: -8
Enter center y: -14
Radius: 86.0, Height: 4.0
Center: (-8.0, -14.0)
Volume: 92940.88
Surface Area: 48631.85
Base Area: 23235.22
```

Experiment-55:

WAP to implement a class STACK with following operations:

a. Check Current Status

b. Push an item

c. Pop an item

d. Get the top item

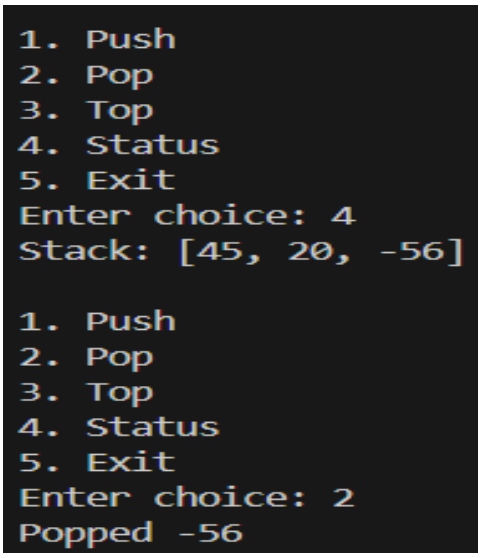
Code 3:

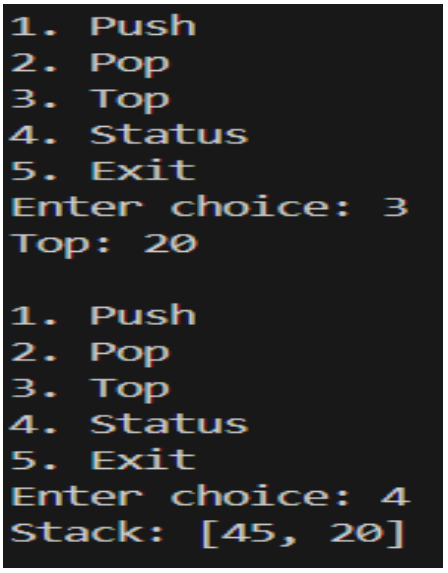
```
class Stack:
    def __init__(self, size=10):
```

```
        self.stack = []
        self.max_size = size

    def push(self, item):
        if len(self.stack) < self.max_size:
            self.stack.append(item)
            print(f"Pushed {item}")
        else:
            print("Stack overflow")
    def pop(self):
        if len(self.stack) > 0:
            item = self.stack.pop()
            print(f"Popped {item}")
            return item
        else:
            print("Stack underflow")
            return None

    def top(self):
        if len(self.stack) > 0:
            return self.stack[-1]
        return None
    def status(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print(f"Stack: {self.stack}")
stack = Stack();while True:
```

```
    print("\n1. Push\n2. Pop\n3. Top\n4. Status\n5. Exit")
    choice = input("Enter choice: ")
    if choice == '1': item = int(input("Enter item:
"));        stack.push(item)
    elif choice == '2': stack.pop()
    elif choice == '3': print(f"Top: {stack.top()}")
    elif choice == '4': stack.status()
    elif choice == '5': break
    else: print("Invalid choice")
```

Output:





Experiment-56: WAP to create a String class and do the following:

a. Write a changeCase() member function that converts all alphabetic characters

present in an input string to lower-case if it is in upper-case and to upper-case

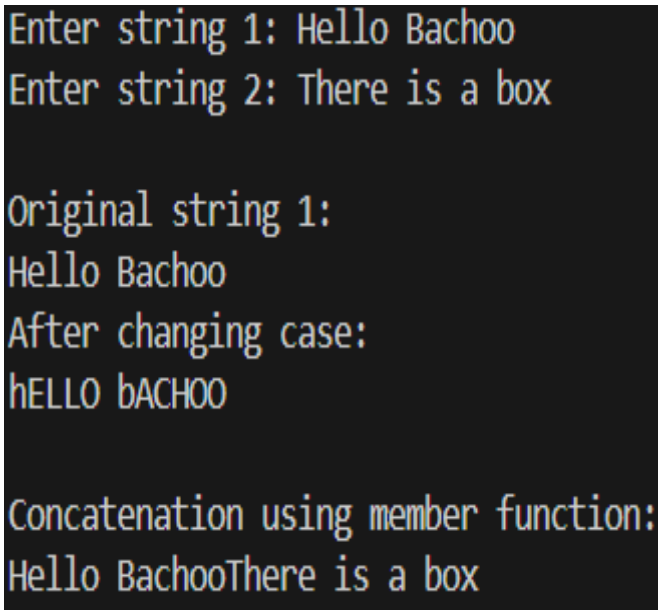
if it is in lower-case.

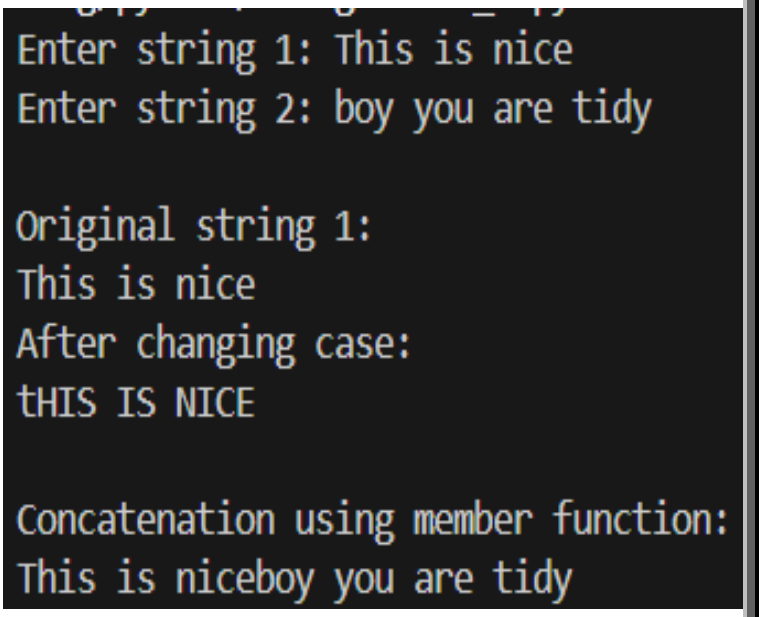
b. Write a concatStringM() member function to concate two strings.

Code 4:

```
class String:
    def __init__(self, s=""):
        self.str = s
    def changeCase(self):
        result = ""
        for char in self.str:
            if char.isupper():
                result += char.lower()
            elif char.islower():
                result += char.upper()
            else:
                result += char
        return result
    def concatStringM(self, other):
        return self.str + other.str
    def display(self):
        print(self.str)
print("After changing case:")
print(str1.changeCase())

print("\nConcatenation using member function:")
str3 = String(str1.concatStringM(str2))
str3.display()

print("Concatenation using friend function:")
result = concatStringF(str1, str2)
print(result)
```

Output:





Experiment-57:

WAP to create a class Point that consists of x and y co-ordinates and using this create a class LineSegment which consists of two points. Write a function compareLines()

to check whether two line segments are parallel or perpendicular or not.

Code 5:

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
class LineSegment:
    def __init__(self, p1, p2):
        self.p1 = p1
        self.p2 = p2
    def slope(self):
        if self.p2.x - self.p1.x == 0:
```

```
            return float('inf')
        return (self.p2.y - self.p1.y) / (self.p2.x - self.p1.x)
def compareLines(line1, line2):
    slope1 = line1.slope()
    slope2 = line2.slope()
    if slope1 == slope2:
        print("Lines are parallel")
    elif slope1 * slope2 == -1:
        print("Lines are perpendicular")
    else:
        print("Lines are neither parallel nor perpendicular")
p1 = Point(float(input("Enter x1: ")), float(input("Enter y1: ")))
p2 = Point(float(input("Enter x2: ")), float(input("Enter y2: ")))
p3 = Point(float(input("Enter x3: ")), float(input("Enter y3: ")))
p4 = Point(float(input("Enter x4: ")), float(input("Enter y4: ")))
line1 = LineSegment(p1, p2)
line2 = LineSegment(p3, p4)
compareLines(line1, line2)
```

Output:

```
Enter x1: 4
Enter y1: -1
Enter x2: 5
Enter y2: 3
Enter x3: 7
Enter y3: -2
Enter x4: 1
Enter y4: -6
Lines are neither parallel nor perpendicular
```

```
Enter x1: 0
Enter y1: 0
Enter x2: 1
Enter y2: 1
Enter x3: 2
Enter y3: 2
Enter x4: 3
Enter y4: 3
Lines are parallel
```

```
Enter x1: 0
Enter y1: 0
Enter x2: 4
Enter y2: 4
Enter x3: 4
Enter y3: 0
Enter x4: 0
Enter y4: 4
Lines are perpendicular
```

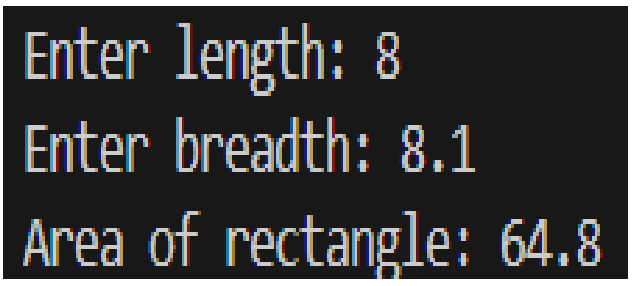
LAB ASSIGNMENT 8

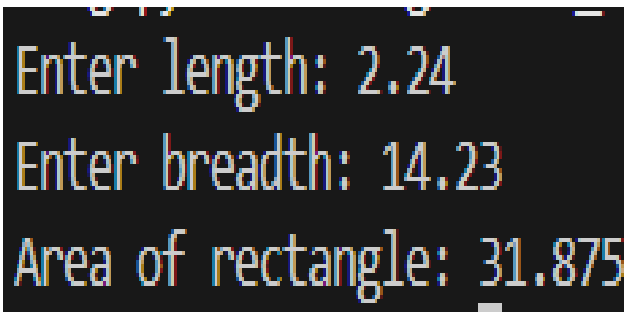
Experiment-58: Write a program to print the area of a rectangle by creating a class named 'Area' having one function. Length and breadth of the rectangle are entered through keyboard.

Code 1:

```
class Area:
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth
    def calculateArea(self):
        return self.length * self.breadth
length = float(input("Enter length: "))
breadth = float(input("Enter breadth: "))
area = Area(length, breadth)
print(f"Area of rectangle: {area.calculateArea()}")
```

Output:





Experiment-59: Write a program to demonstrate ATM money withdrawal and deposit process by taking following private data members: Accountno, balance; Accountno and balance data member initialize using __init__ Write three functions

1. Deposit 2. Withdraw 3. Balance

Write menu driven choice

1. Deposit

2. Withdraw

3. Balance

4. Exit

Program stop execution when user enter choice 4.

Code 2:

```
class ATM:
    def __init__(self, account_no, balance):
        self.__account_no = account_no
        self.__balance = balance
    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited: {amount}. New balance:
{self.__balance}")
    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn: {amount}. New balance:
{self.__balance}")
        else:
            print("Insufficient balance")
```

```
    def checkBalance(self):
        print(f"Account No: {self.__account_no}, Balance:
{self.__balance}")
account_no = input("Enter account number: ")
initial_balance = float(input("Enter initial balance: "))
atm = ATM(account_no, initial_balance)
while True:
    print("\n1. Deposit\n2. Withdraw\n3. Balance\n4. Exit")
    choice = input("Enter choice: ")
    if choice == '1':
        amount = float(input("Enter amount to deposit: "))
        atm.deposit(amount)
    elif choice == '2':
        amount = float(input("Enter amount to withdraw: "))
        atm.withdraw(amount)
    elif choice == '3':
        atm.checkBalance()
    elif choice == '4':
        print("Thank you for using ATM")
        break
    else: print("Invalid choice")
```

Output:

```
1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 3
Account No: 9548373611, Balance: 4800.0

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 4
Thank you for using ATM
```

```
Enter account number: 9548373611
Enter initial balance: 5000

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 1
Enter amount to deposit: 200
Deposited: 200.0. New balance: 5200.0

1. Deposit
2. Withdraw
3. Balance
4. Exit
Enter choice: 2
Enter amount to withdraw: 400
Withdrawn: 400.0. New balance: 4800.0
```

Experiment-60: Create a class “Mobile” with attributes: brand, price, color, width, height. Use __init__ to set default values of these attributes. Write function to display details of

all attributes.

Code 3:

```
class Mobile:

    def __init__(self, brand="", price=0, color="", width=0, height=0):

        self.brand = brand

        self.price = price

        self.color = color

        self.width = width

        self.height = height

    def display(self):

        print(f"Brand: {self.brand}, Price: {self.price}, Color: {self.color}, Width: {self.width}, Height: {self.height}")

mobile = Mobile(input("Enter brand: "), float(input("Enter price: ")), input("Enter color: "), float(input("Enter width: ")), float(input("Enter height: ")))

mobile.display()
```

Output:

```
Enter brand: Samsung
Enter price: 23200
Enter color: Magenta
Enter width: 10
Enter brand: IPhone
Enter price: 54201
Enter color: Silver
Enter width: 8
Enter height: 10
Brand: IPhone, Price: 54201.0, Color: Silver, Width: 8.0, Height: 10.0
```

Experiment-61: Create a class “Mobile” with attributes: brand, price, color. Enter detail of five different mobile. (Using Array of object).

a) Display total number of mobile having price greater than 5000

b) Display Brand, Price and color for all mobiles for price range 1000 to 10000

Code 4:

```
class Mobile:
    def __init__(self, brand="", price=0, color=""):
        self.brand = brand
        self.price = price
        self.color = color
mobiles = [];for i in range(5):
    print(f"\nEnter details for mobile {i+1}:")
    brand = input("Enter brand: ")
    price = float(input("Enter price: "))
    color = input("Enter color: ")
    mobiles.append(Mobile(brand, price, color))
count_above_5000 = sum(1 for m in mobiles if m.price > 5000)
print(f"\nTotal mobiles with price > 5000: {count_above_5000}")
print("\nMobiles with price between 1000 and 10000:")
for m in mobiles:
    if 1000 <= m.price <= 10000:
      print(f"Brand: {m.brand}, Price: {m.price}, Color:{m.color}")
```

Output :

```
Enter details for mobile 1:
Enter brand: Samsung
Enter price: 6600
Enter color: Violet

Enter details for mobile 2:
Enter brand: Redmi
Enter price: 4999
Enter color: Red

Enter details for mobile 3:
Enter brand: Nokia
Enter price: 3400
Enter color: Black

Enter details for mobile 4:
Enter brand: IPhone
Enter price: 20000
Enter color: Silver

Enter details for mobile 5:
Enter brand: Samsung
Enter price: 21200
Enter color: White

Total mobiles with price > 5000: 3

Mobiles with price between 1000 and 10000:
Brand: Samsung, Price: 6600.0, Color: Violet
Brand: Redmi, Price: 4999.0, Color: Red
Brand: Nokia, Price: 3400.0, Color: Black
```